Chapter 3
Memory Management

3.1 No Memory Abstraction

3.2 A Memory Abstraction: Address Space

3.3 Virtual Memory

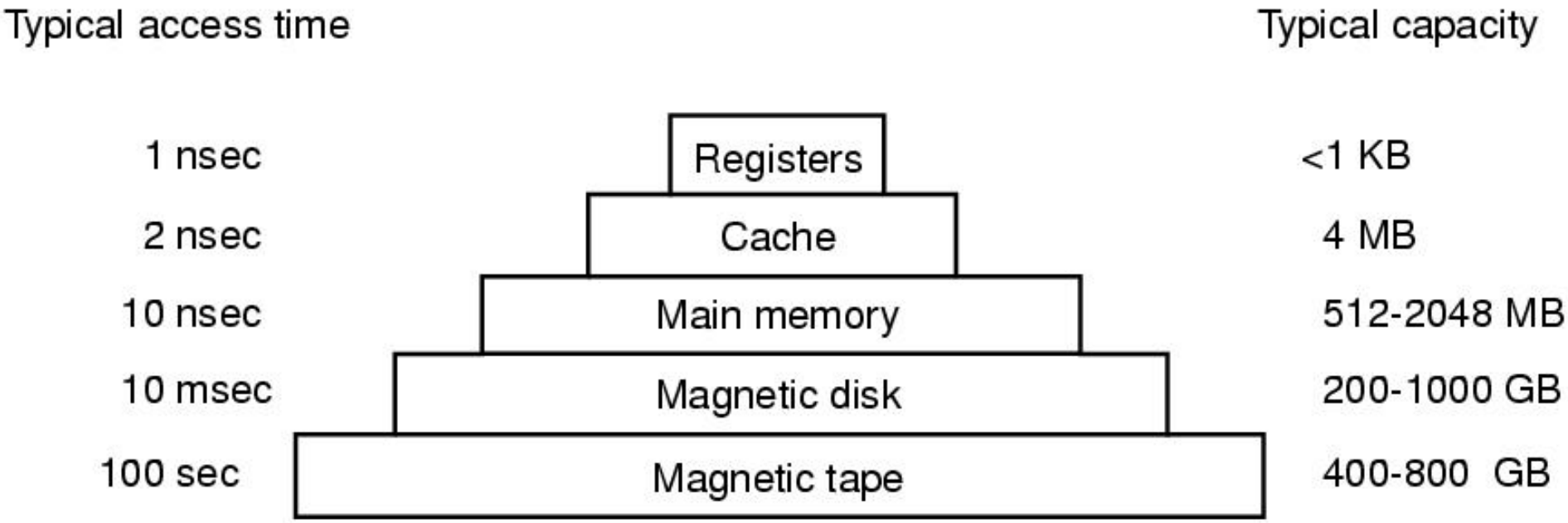
3.4 Page Replacement Algorithm

3.5 Segmentation

Storage Hierarchy

- Ideally programmers want memory that is
  - Large
  - Fast
  - Nonvolatile
  - Inexpensive
- Memory hierarchy
  - A few MBs of fast, expensive, volatile cache memory
  - A few GBs of medium-speed, medium price, volatile main memory (RAM)
  - A few TBs of slow, cheap, nonvolatile disk storage
- Memory manager handles the memory hierarchy.

Storage Hierarchy



Memory

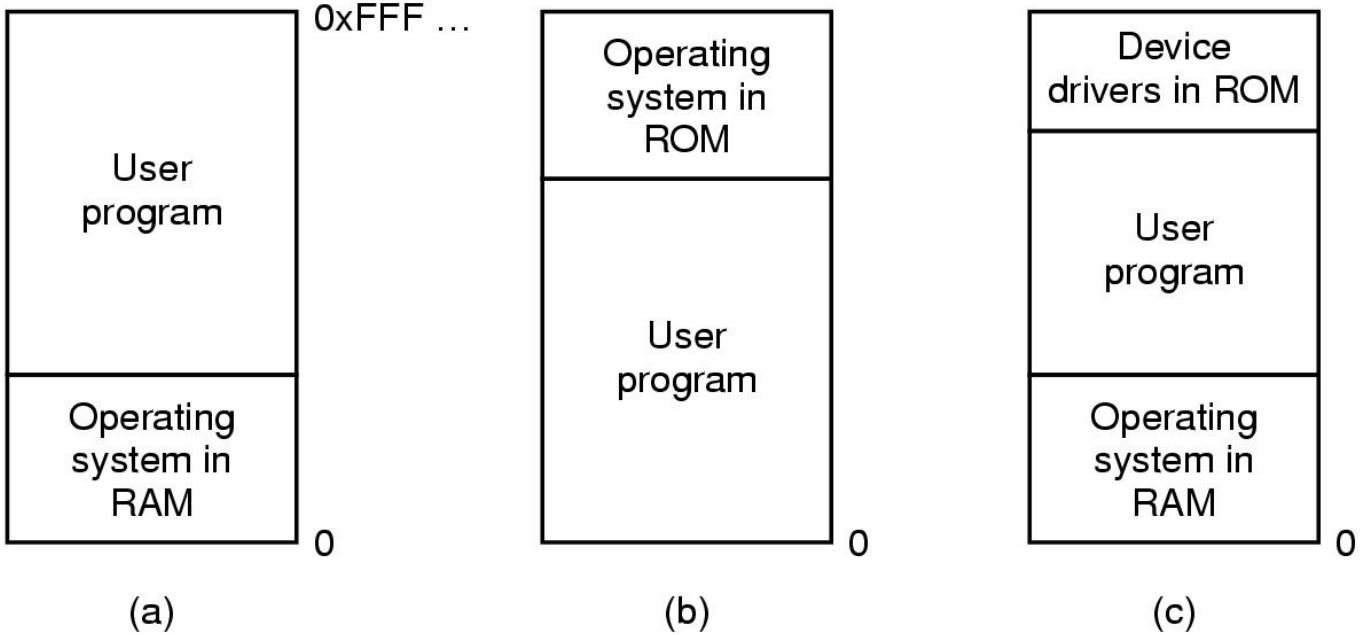
- Program must be brought into memory and placed within a process for it to be run.
- Consecutive address space (memory unit)，is used for store the code and data of process.
- System section and User section

Objective

1. Support multiprogramming
2. Convenience to user, Hide hardware details to user program, Auto-load user program
3. Solve program space > memory space
4. Process can be flexible in memory
5. Quick access
6. Storage protection and security
7. Share and communication
8. Performance and cost

No Memory Abstraction

- The simplest memory abstraction is no memory abstraction. Every program simply saw the physical memory.
- It is not possible to run two programs in memory at the same time.
- Three different ways to organize memory with OS and one user program.



Multiple Programs without Memory Abstraction

- IBM 360
  - Divide memory into 2-KB blocks
  - Assign each block a 4-bit protection key, which saved in special registers of CPU
  - PSW also contains a 4-bit key
  - When a process access memory, compare the protection code of the memory block with the PSW key.

Multiple Programs without Memory Abstraction

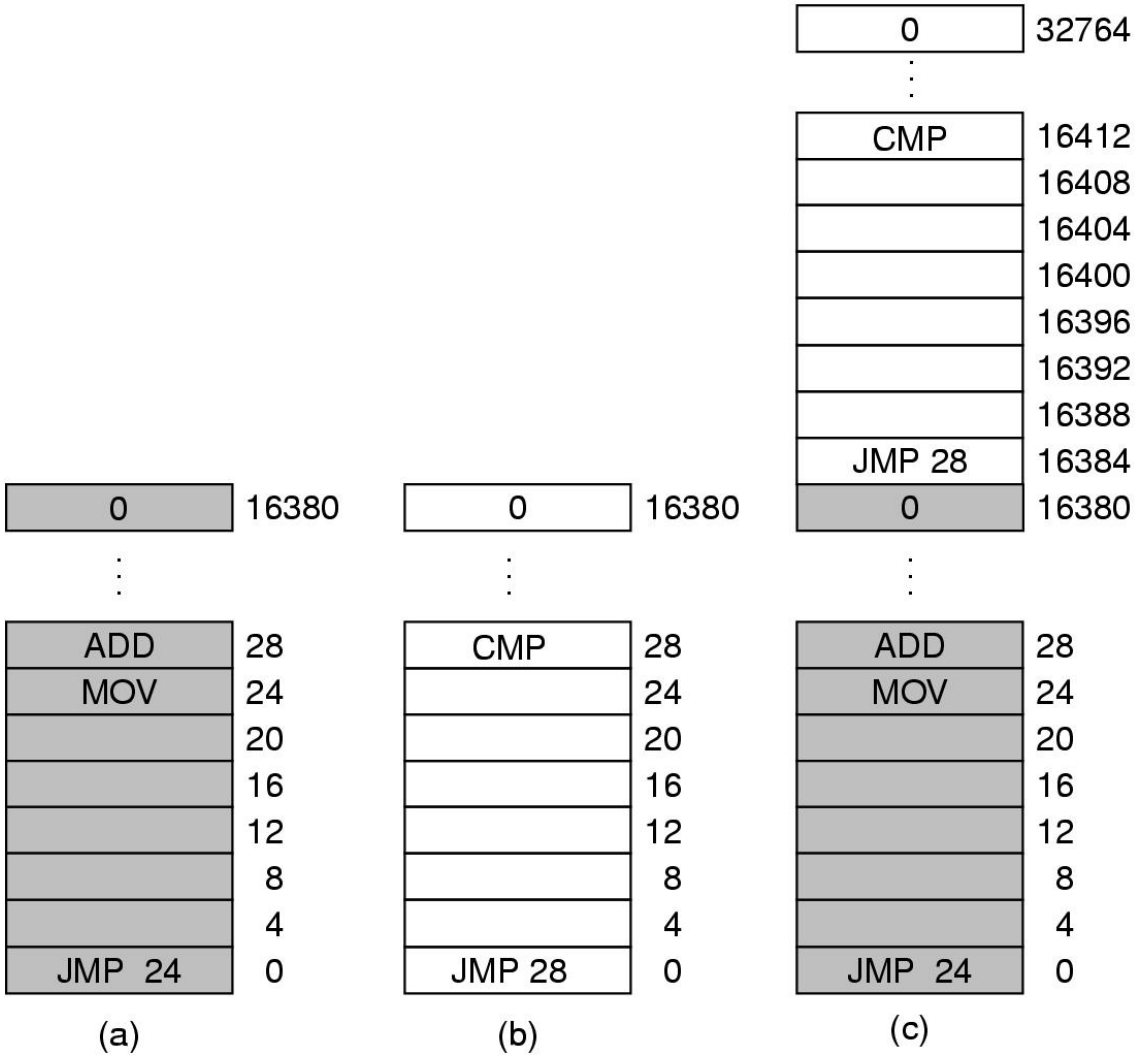


Illustration of the relocation problem.

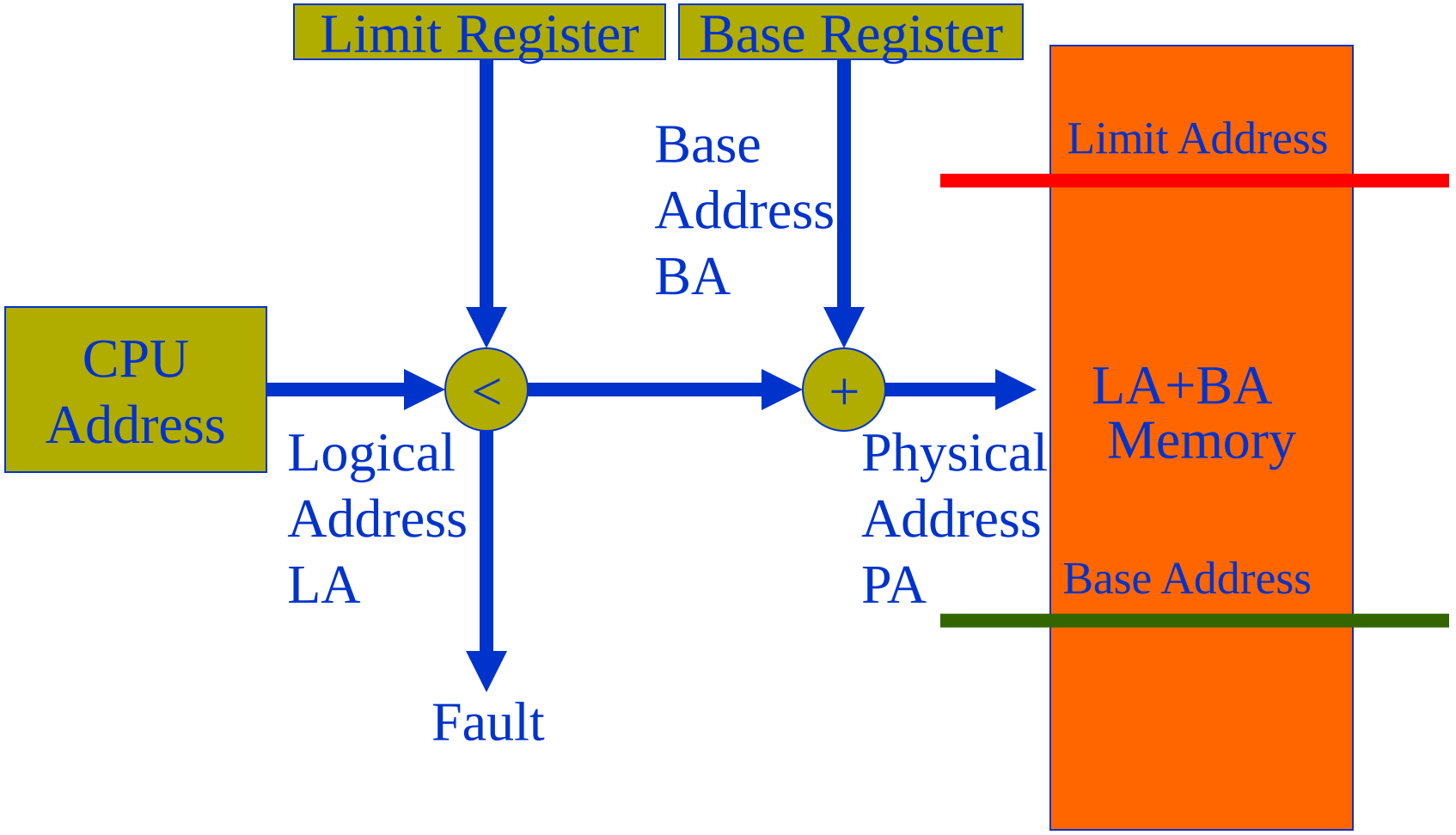
Address Spaces

- To allow multiple applications to be in memory at the same time without interfering with each other, two problems must be solved:
  - Protection
    - Primitive solution: Label chunks of memory with a protection key, and compare the key of the executing process to that of every memory word fetched.
  - Relocation
- A better solution: **Address Space**
  - Address space is a set of addresses that a process can use to address memory.

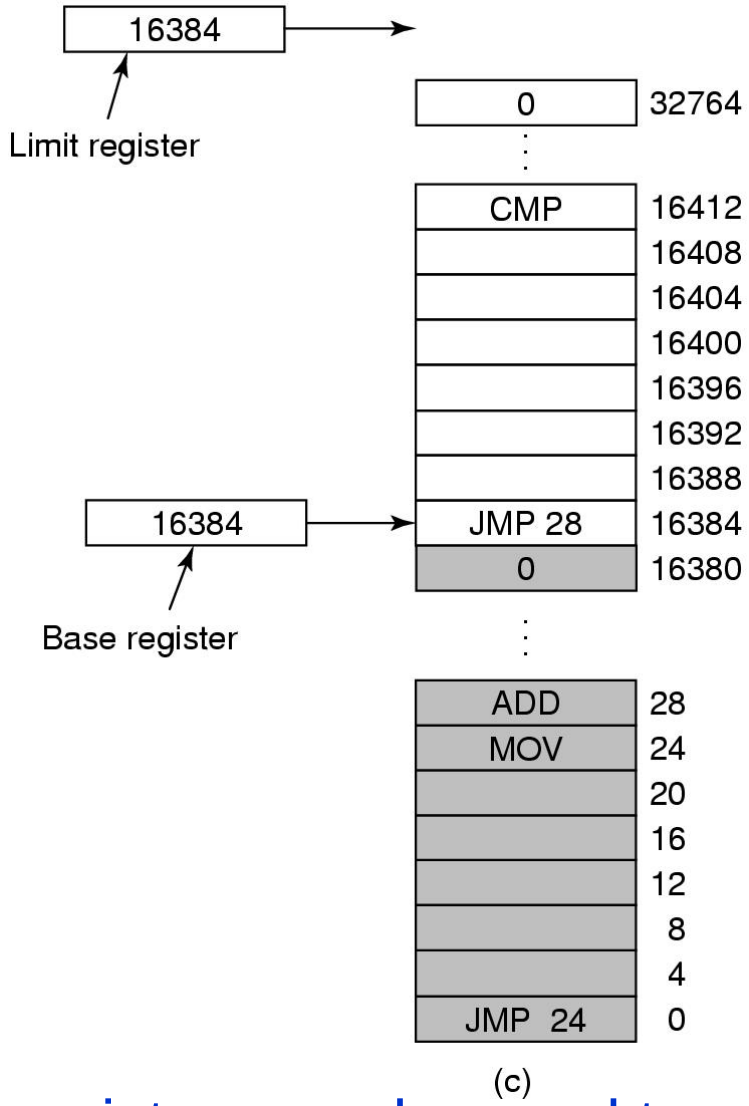
Address Spaces

- Each process has its own address space, independent of those belonging to other processes.
  - Logical address space, range (0 to max)
    - Logical addresses, Virtual addresses
- Dynamic Relocation
  - Map each process' address space onto a different part of physical memory.
  - Physical address space, range (R+0 to R+max) for base value R
    - Physical addresses, Real addresses
  - Base and Limit Registers

Base and Limit Registers



Base and Limit Registers



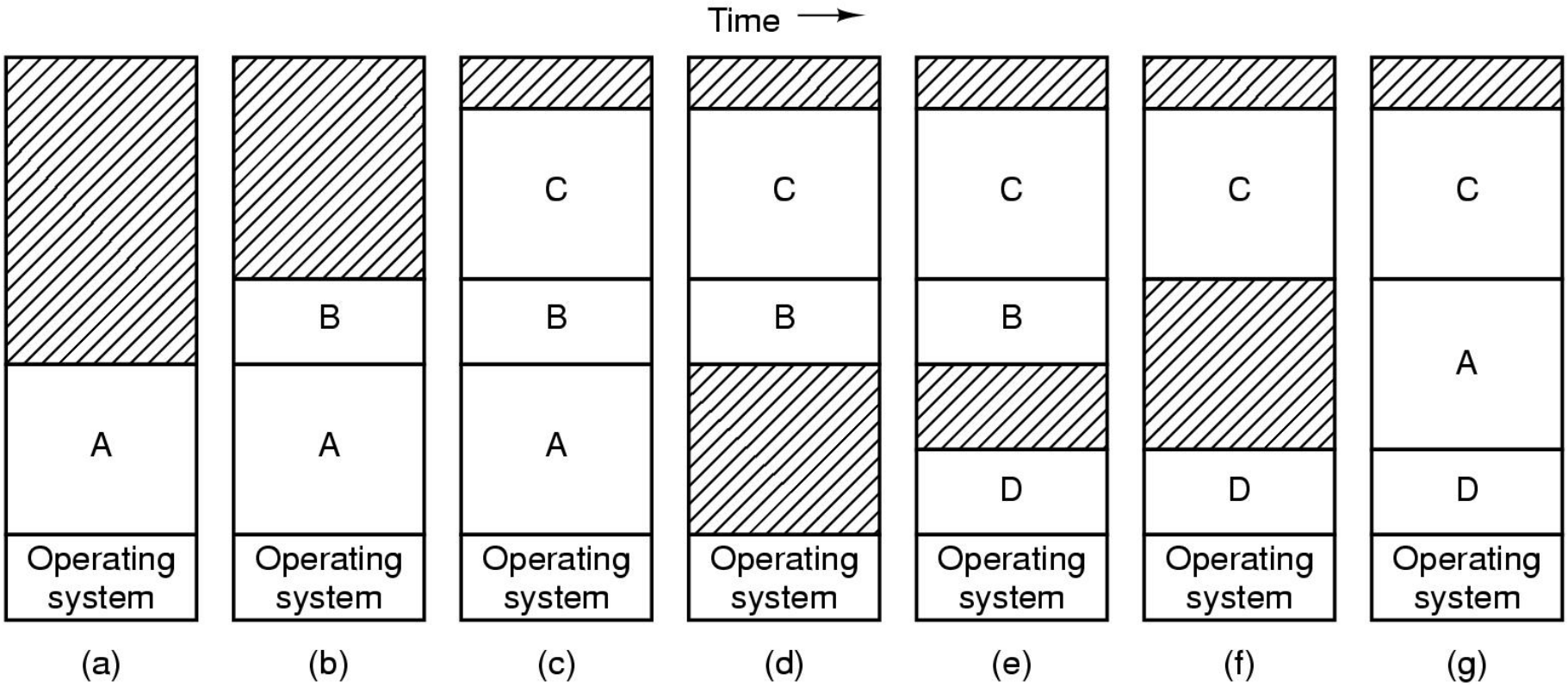
(c)

Base and limit registers can be used to give each process a separate address space.

Swapping (1)

- Lots of programs, total size exceeds memory
  - **Swapping**: brings the whole process into memory, runs it for a while, and then move it back on the disk.
  - **Virtual memory**: allows programs to run even when they are only partially in main memory.
- Swapping allows several processes to share a partition
- Processes that grow can be swapped out and swapped back in a bigger partition

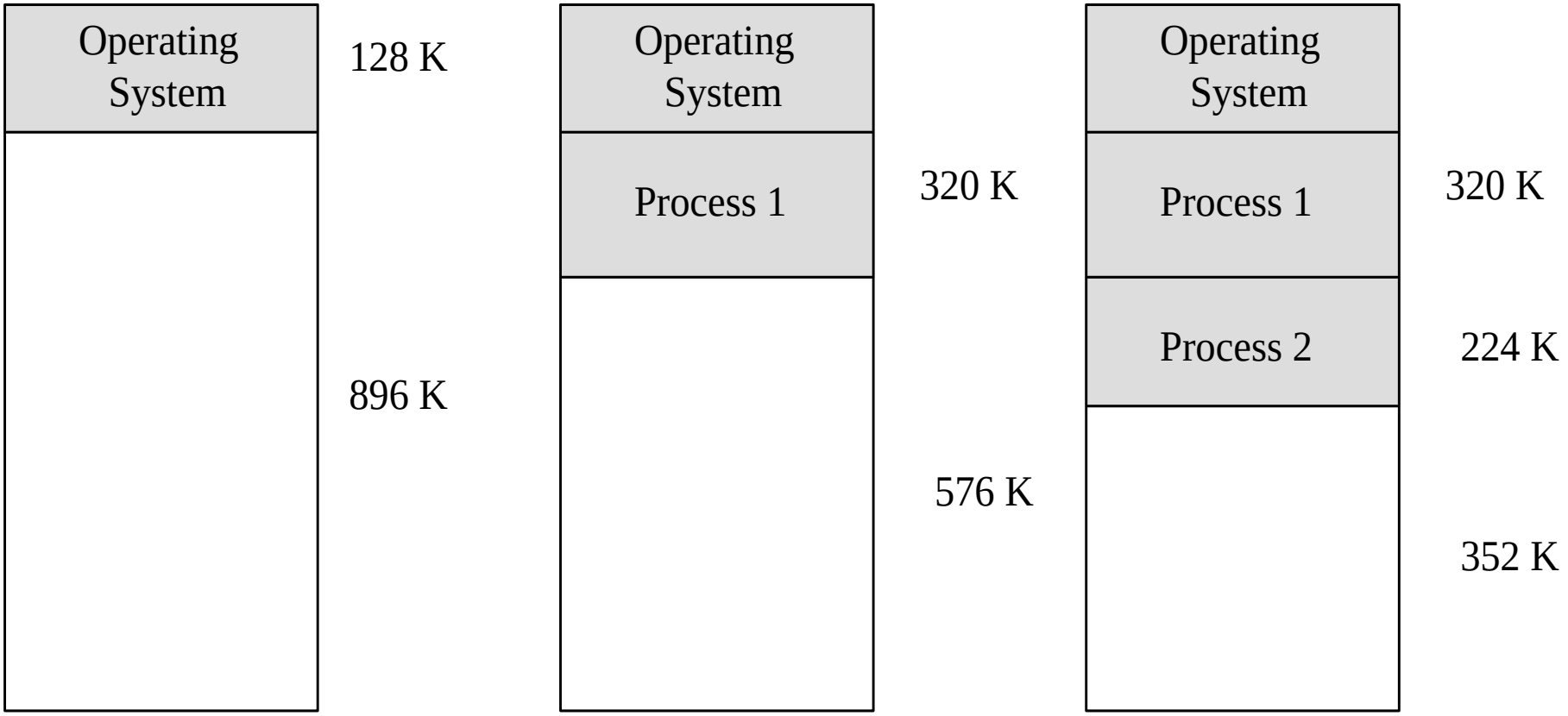
Swapping (2)

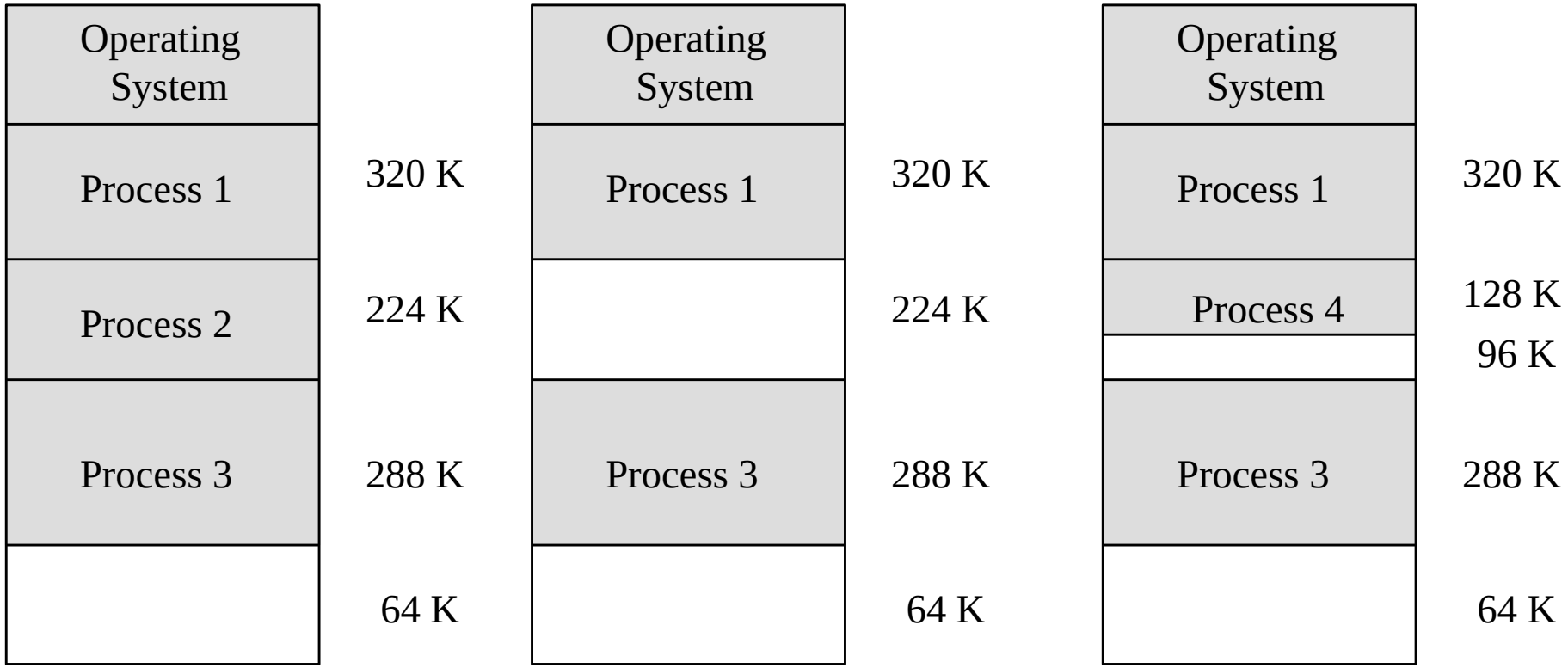


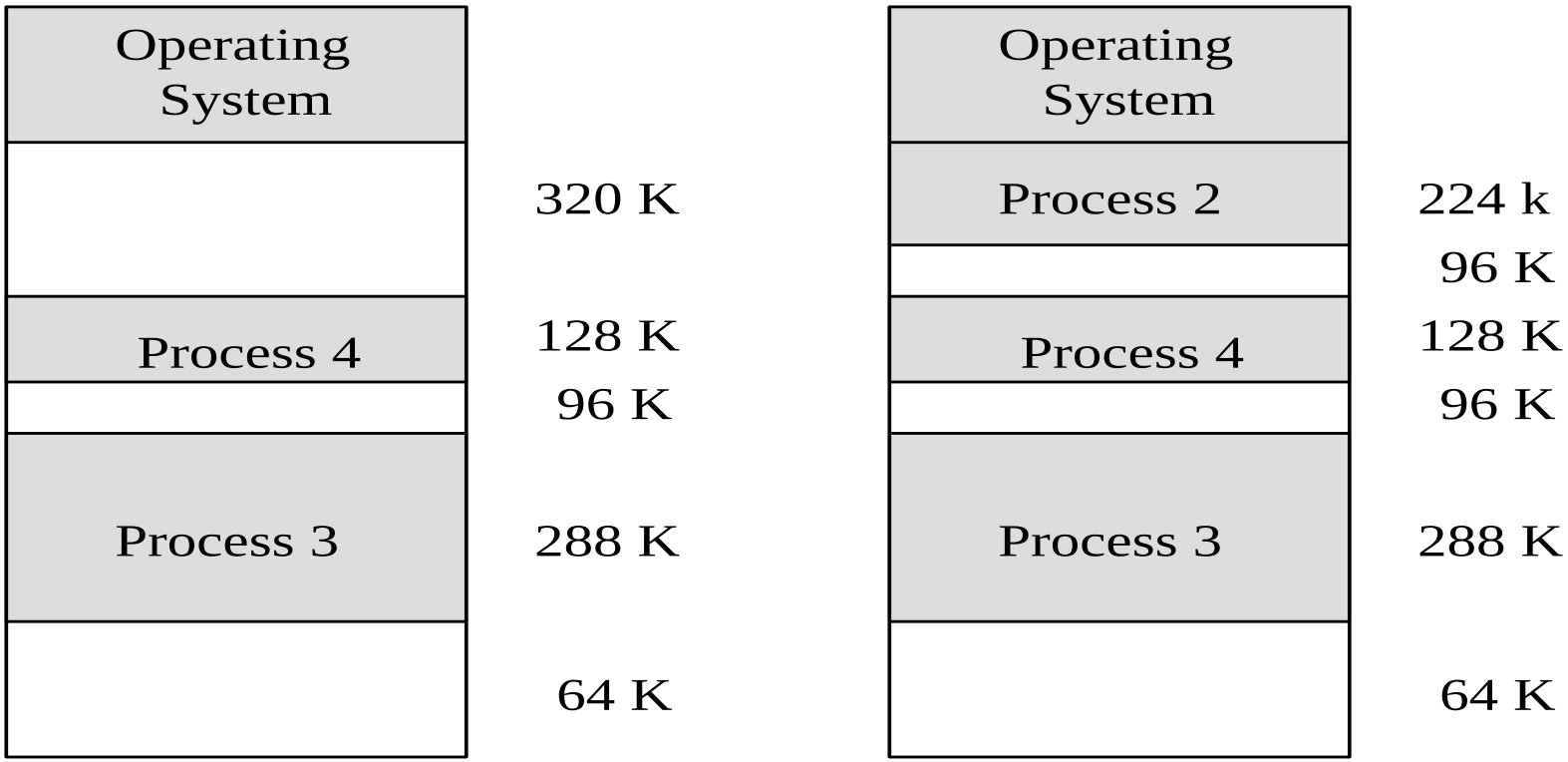
Memory allocation changes as processes come into memory and leave it. The shaded regions are unused memory.

Swapping (3)

- Allocating memory dynamically
  - When a process come, allocate space as it demand if have enough space;
  - Or process wait on disk;
- With processes swapping in and swapping out, there will be many **holes** (External Fragmentation)


Operating System
128 K
896 K
Operating System
Process 1
320 K
576 K
Operating System
Process 1
320 K
Process 2
224 K
352 K


Operating System
Process 1
320 K
Process 2
224 K
Process 3
288 K
64 K
Operating System
Process 1
320 K
224 K
Process 3
288 K
64 K
Operating System
Process 1
320 K
Process 4
128 K
96 K
Process 3
288 K
64 K

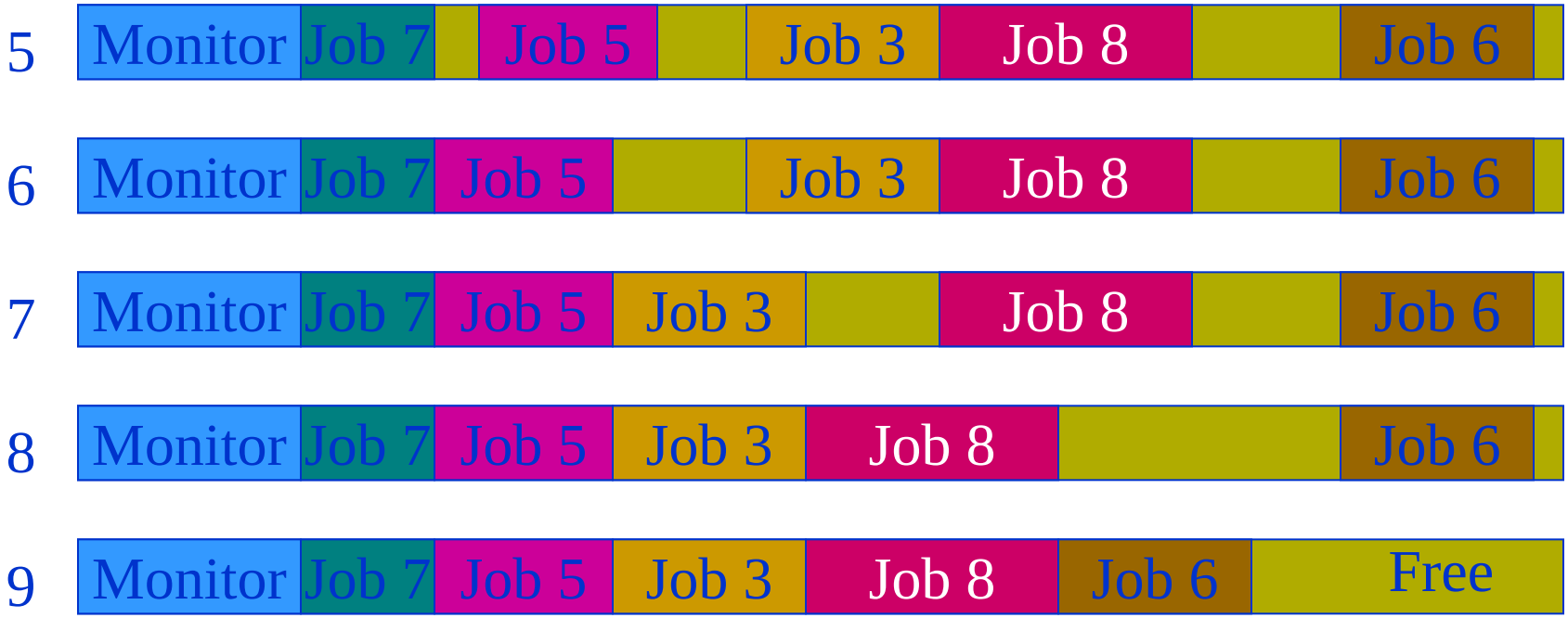


- The holes need be compacted together to generate a large free space.

Compaction

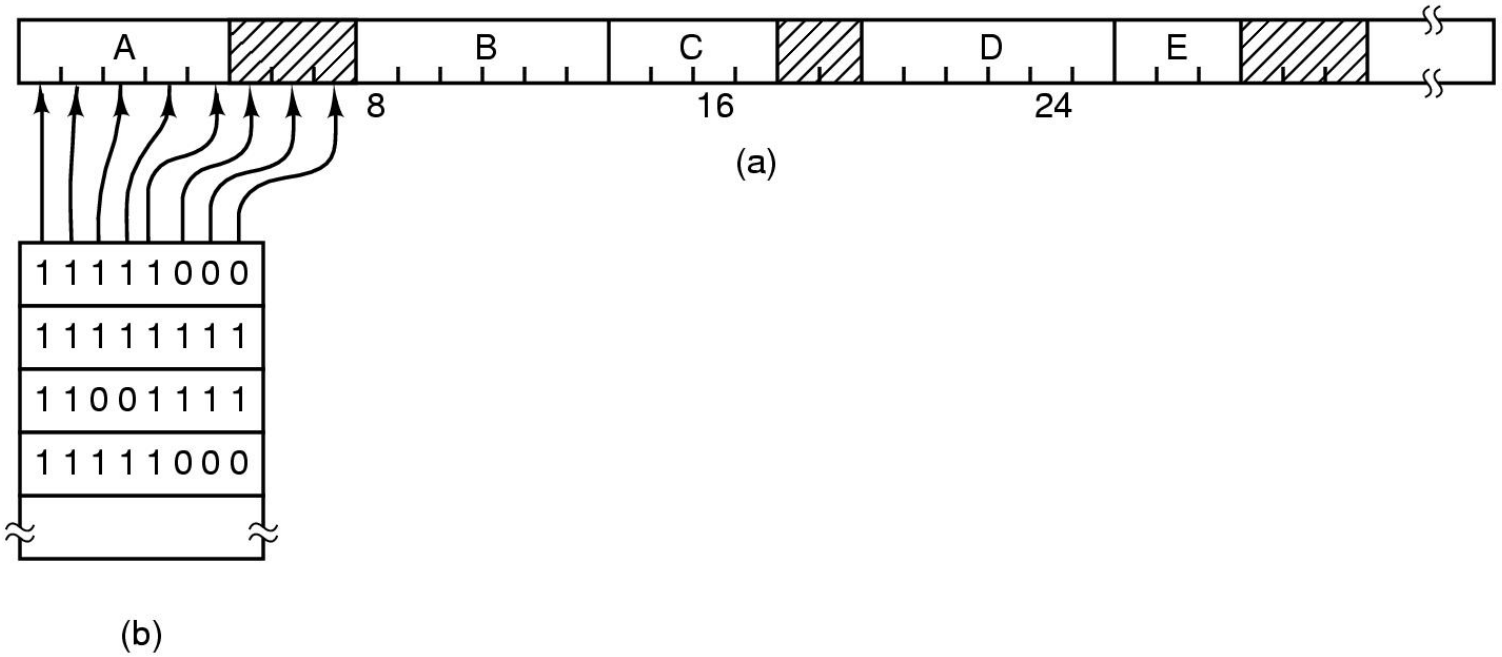
(Similar to Garbage Collection)

- Assumes programs are all re-locatable
- Processes must be suspended during compaction
- Need be done only when fragmentation gets very bad



Memory Management with Bitmaps

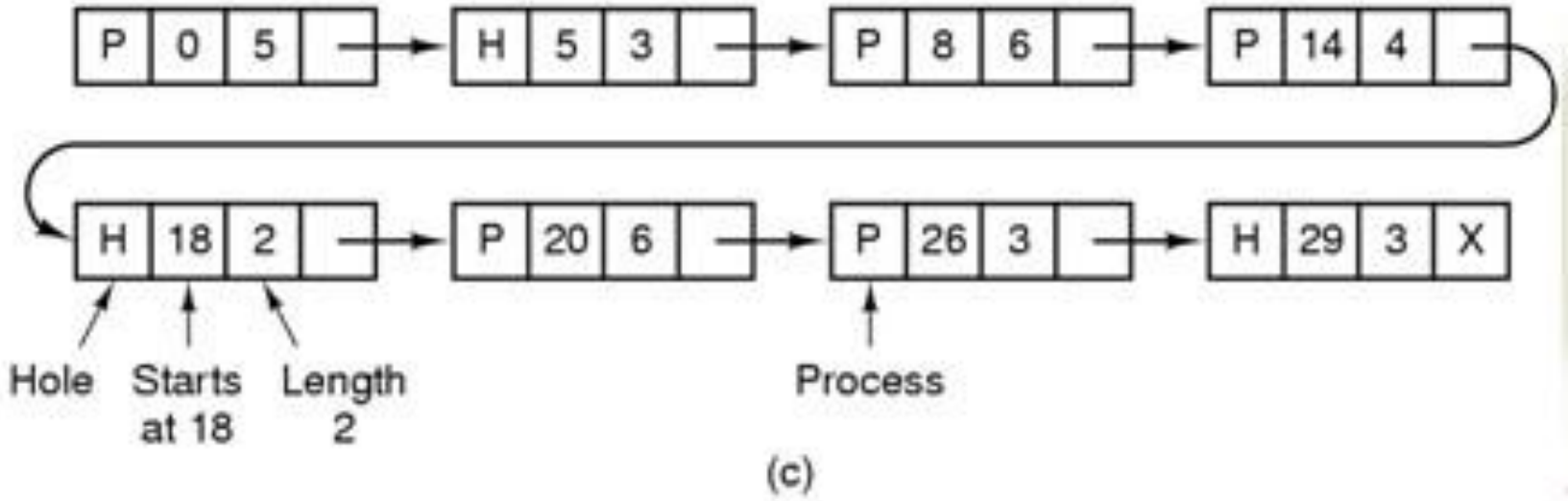
- Use bitmaps or free lists.
- Memory is divided into allocation units.
  - One allocation unit corresponds to 1bit in the bitmap
  - 0: free, 1: allocated



- Size of allocation unit
  - The smaller the allocation unit, the larger the bitmap.
- Problem: allocation
  - When a new process arrives, the manager must find consecutive 0 bits in the map.
  - Searching a bitmap for a run of a given length is a slow operation.

Memory Management with Linked Lists

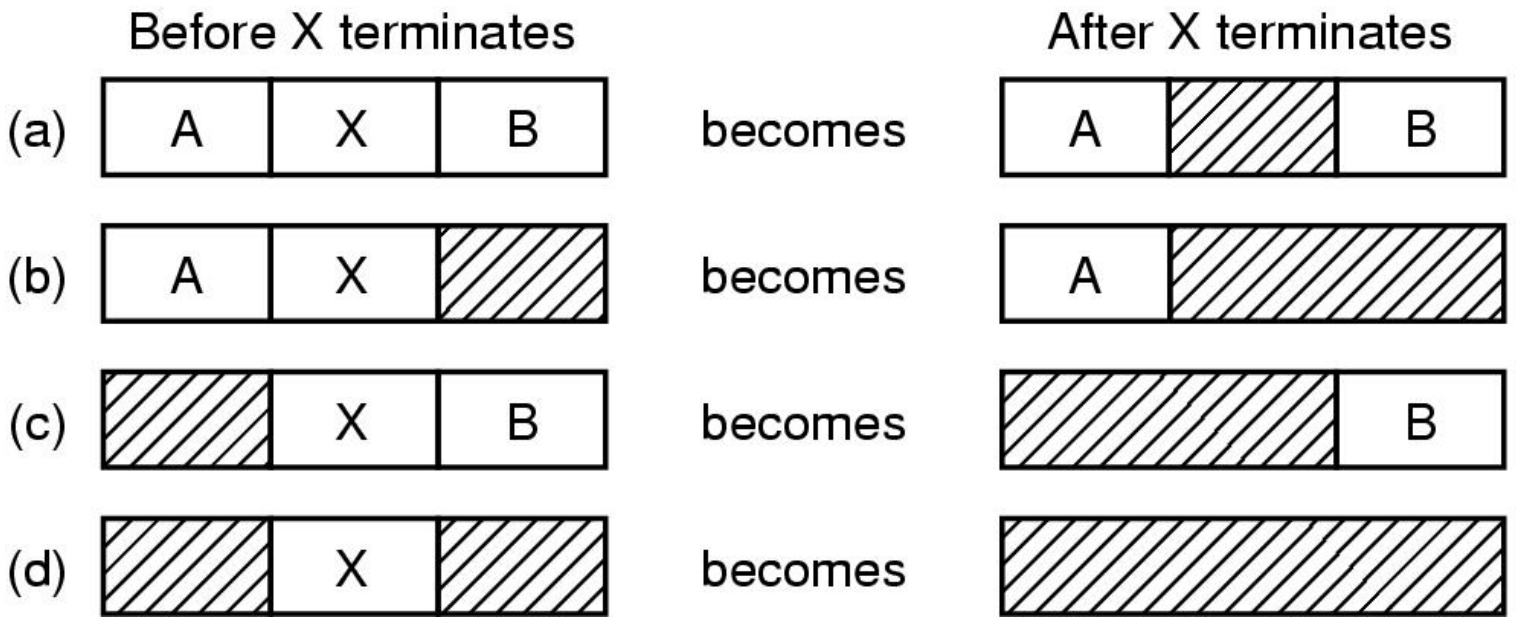
- Use a linked list of allocated and free memory segments (called hole)
  - sorted by the address or by the size



(c)

Memory Management with Linked Lists

- Sorting by address has the advantage:
  - When a process terminates or is swapped out, updating the list is straightforward.



Four neighbor combinations for the terminating process X

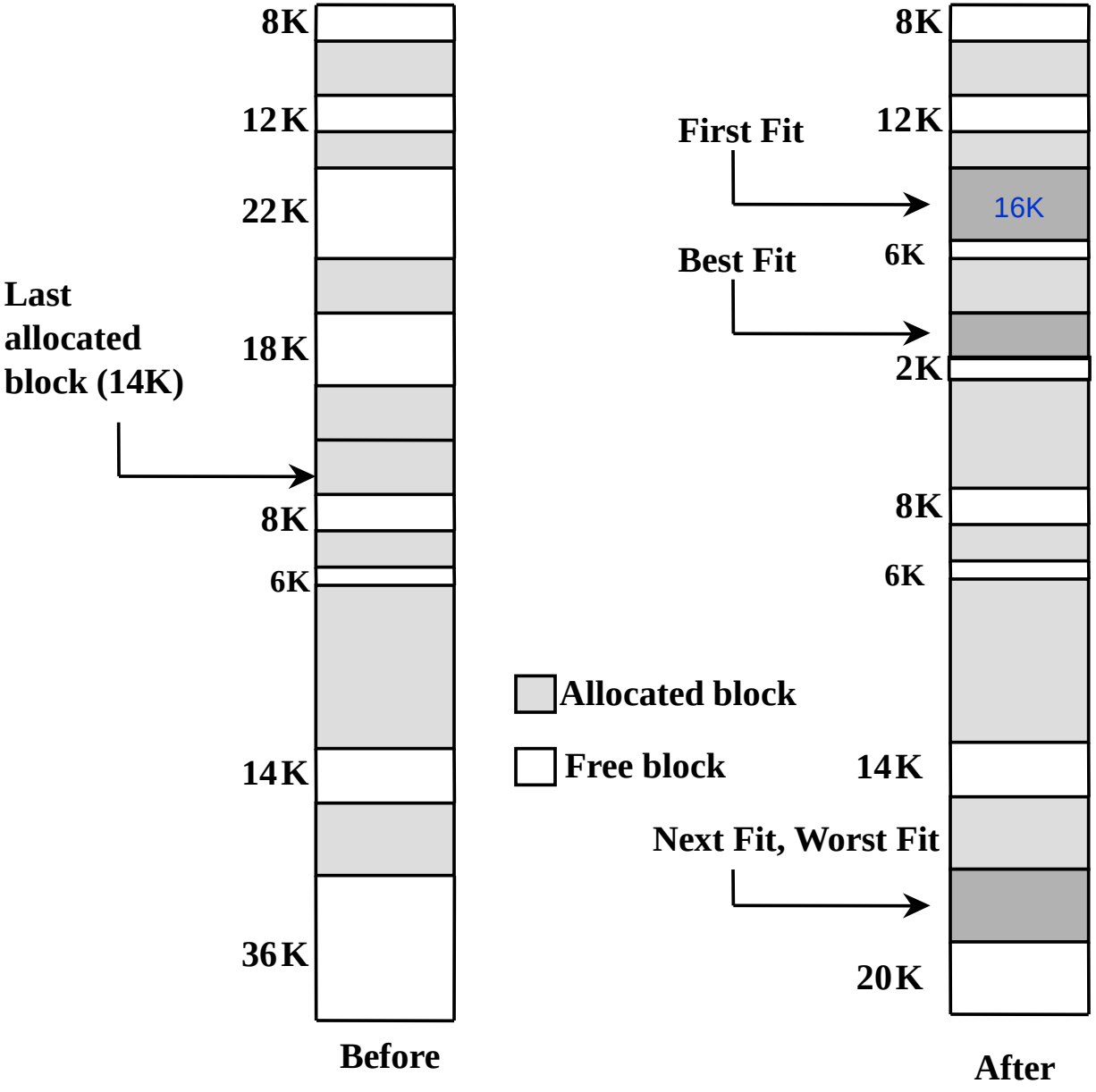
Storage Placement Strategies

How to satisfy a request of size *n* from a list of free holes.

- **First Fit**
  - Use the first available hole whose size is sufficient to meet the need.
  - Problem: Creates average size holes.
- **Next Fit**
  - Minor variation of first fit: search for the last hole stopped.
  - Problem: slightly worse performance than first fit.

Storage Placement Strategies

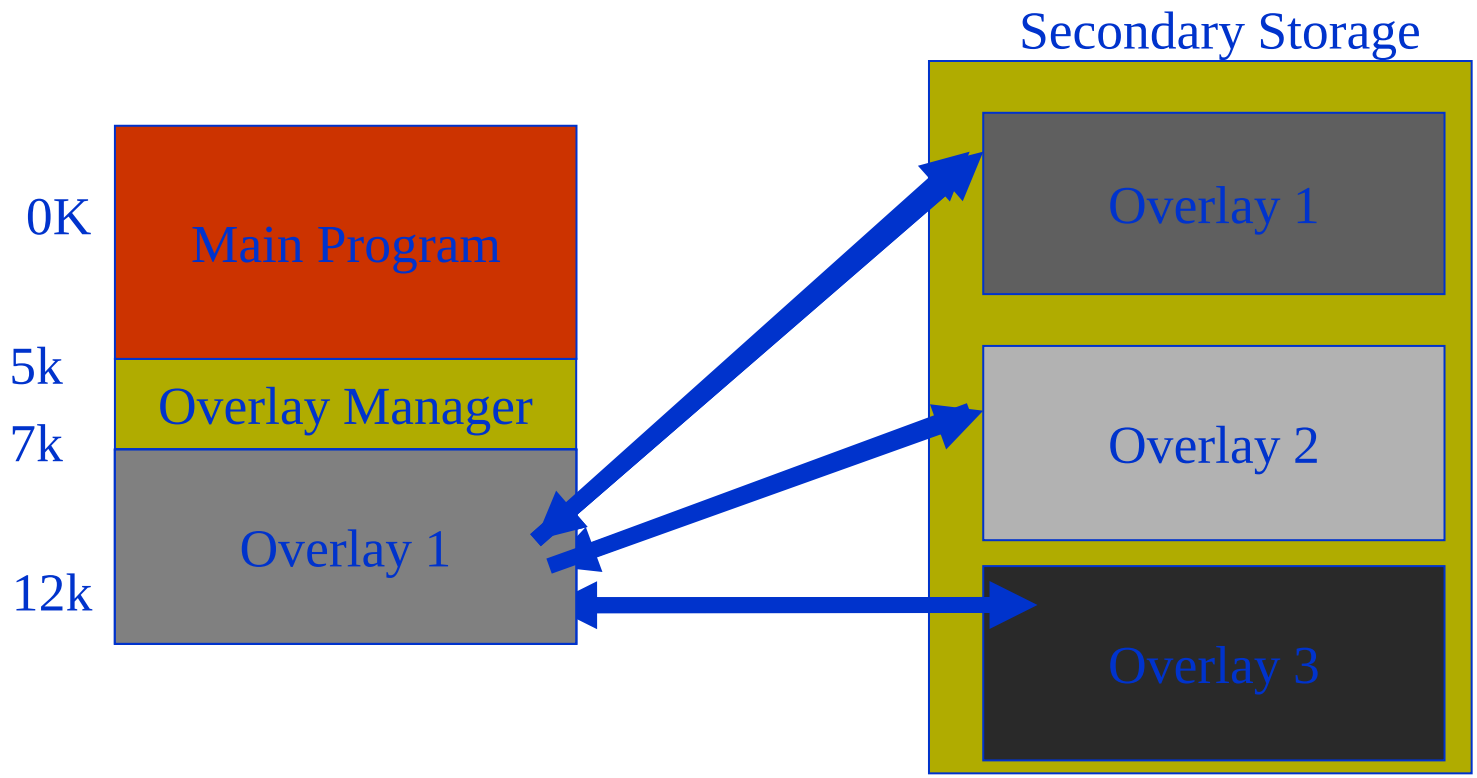
- **Best Fit**
  - Use the hole whose size is equal to the need, or if none is equal, the hole that is larger but closest in size.
  - Problem: Creates small holes that can't be used.
- **Worst Fit**
  - Use the largest available hole.
  - Problem: Gets rid of large holes making it difficult to run large programs.
- **Quick Fit**
  - maintains separate lists for some of the more common sizes requested.
  - When a request comes for placement it finds the closest fit.
  - This is a very fast scheme, but a merge is expensive.


8K
12K
22K
18K
8K
6K
14K
36K
Last allocated block (14K)
Before
First Fit
Best Fit
Next Fit, Worst Fit
16K
Allocated block
Free block
After

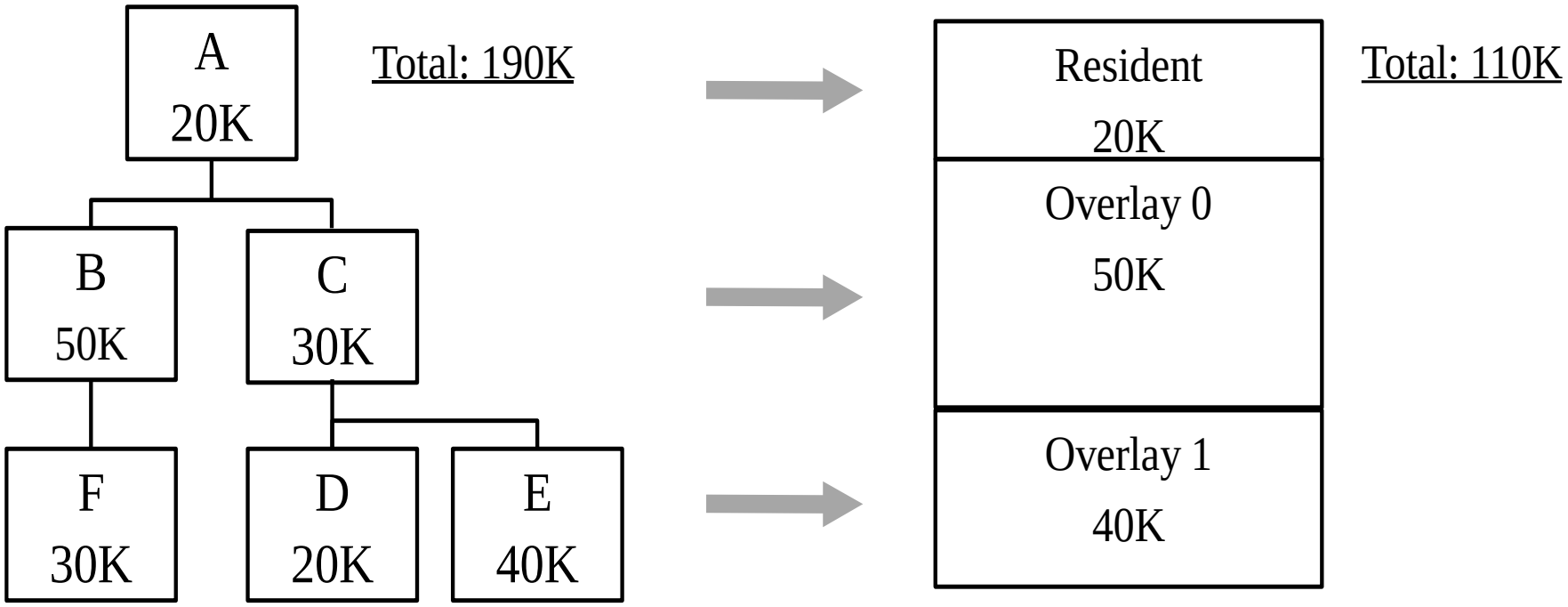
Overlaying

- One program, using lots of memory
  - Overlaying (1960s): split programs into little pieces, called overlays.
  - Allow one or some overlays in memory at the same time, many overlays can use common memory space. (see next page)
  - The work of swapping overlays in and out is done by OS, but the programmer should split the program into pieces manually, which increases the complexity of programming.

Overlaying


Secondary Storage
0K
Main Program
5k
Overlay Manager
7k
Overlay 1
12k
Overlay 1
Overlay 2
Overlay 3

Overlaying Example



An alternative approach：(100K)

- A(20K)： 20K；
- B(50K)、D(20K)、E(40K)： 50K；
- C(30K)、F(30K)： 30K；

Virtual Memory

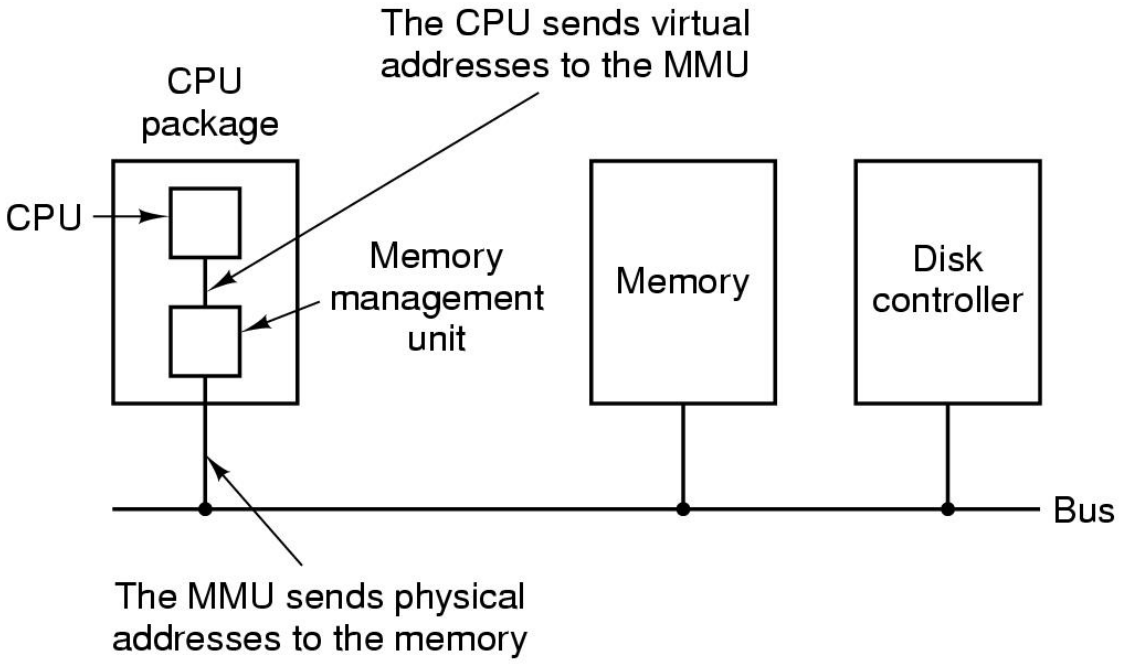
- Virtual memory – separation of user logical memory from physical memory.
  - Provide user with virtual memory that is as big as user needs
  - Logical address space can therefore be much larger than physical address space.
  - Store virtual memory on disk
  - Only part of the program needs to be in memory for execution.
  - Allows address spaces to be shared by several processes.
  - Allows for more efficient process creation.
  - Load and store cached virtual memory without user program intervention

Principle of Locality

- Locality of reference:
  - During the phase of execution the process references relatively small fraction of its pages.
  - Time locality
  - Space locality

Basic idea of Virtual Memory

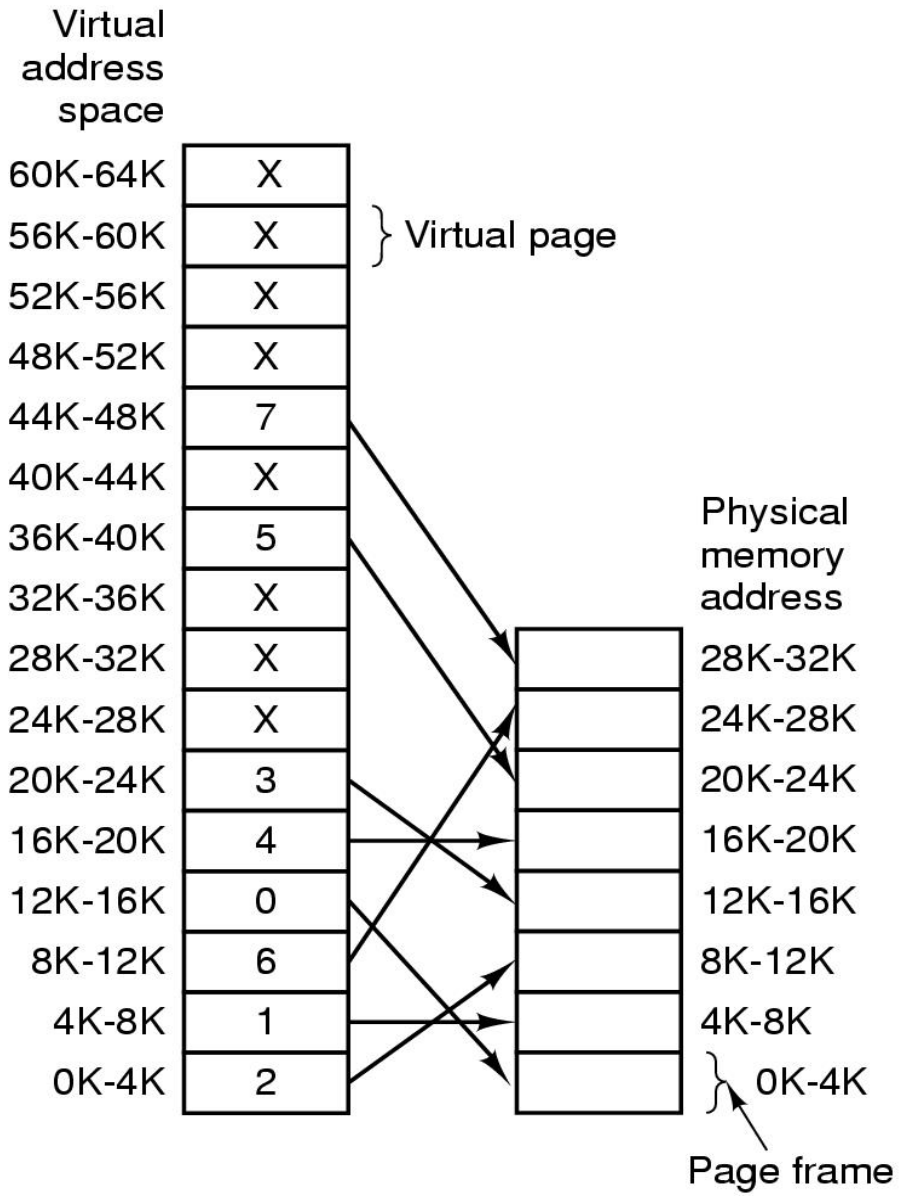
- Paging is a technique used to implement virtual memory.
  - Pages: the virtual address is divided up into units .
  - Page frames: the corresponding units in the physical memory.
- The **MMU** (memory management unit) translates a virtual address into a physical address



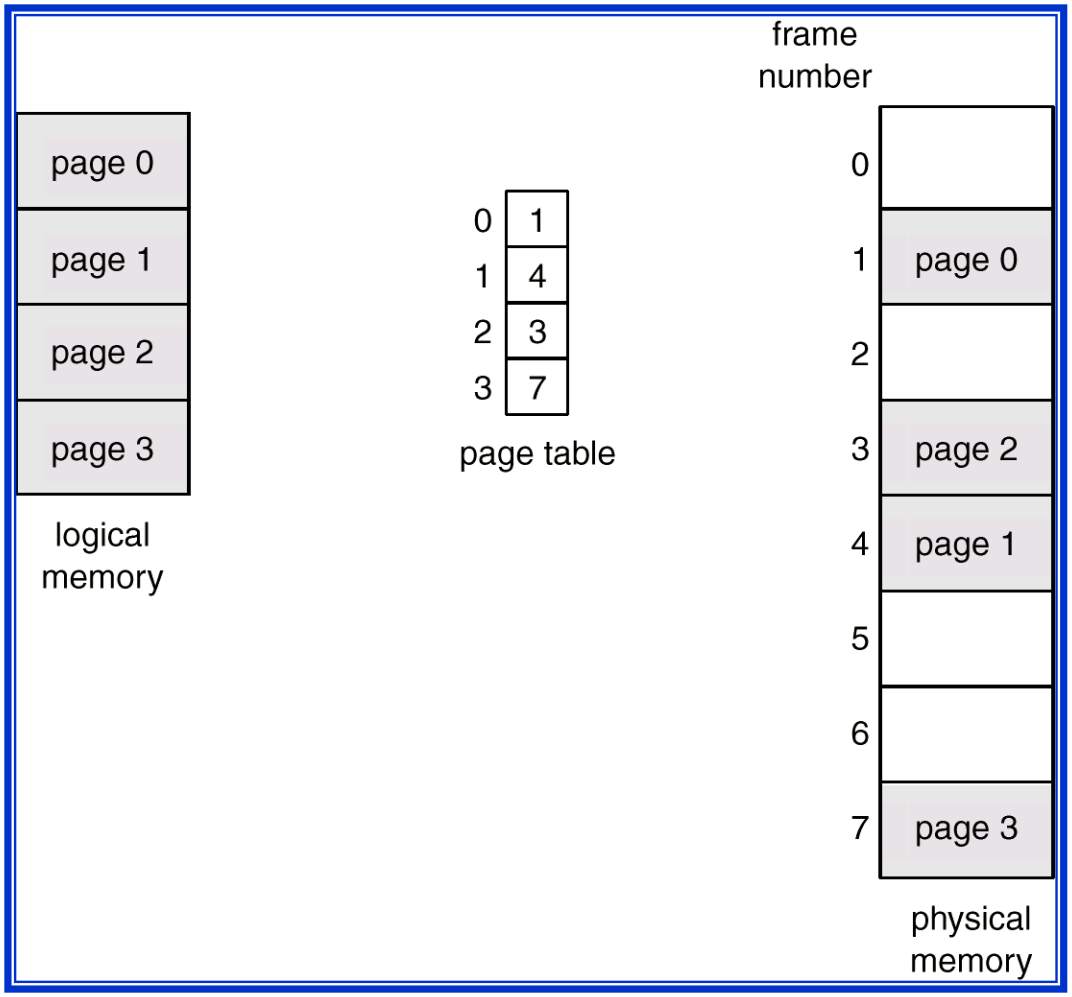
- Suppose the computer can generate 16-bit addresses, (0-64k). However, the computer only has 32k of memory ➔ 64k program can be written, but not loaded into memory.
- A Present/Absent bit keeps track of whether or not the page is mapped.
- Reference to an unmapped page causes the CPU to trap to the OS.
- This trap is called a Page fault. The MMU selects a little used page frame, writes its contents back to disk, fetches the page just referenced, and restarts the trapped instruction.

Paging

- The relation between virtual addresses and physical memory addresses given by **page table**.



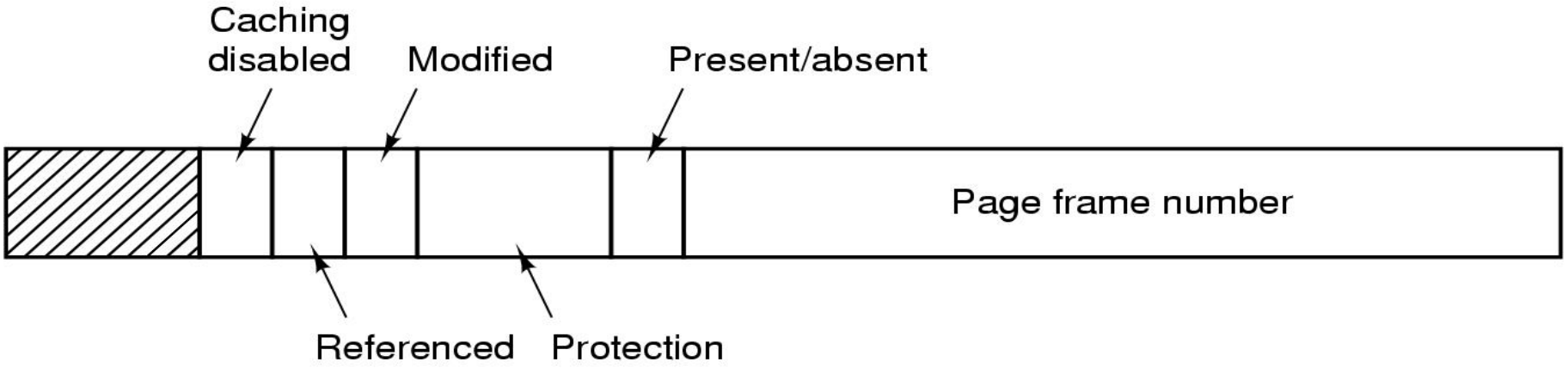
Paging Model Example



Page Table

- Virtual address: Virtual page number (high-order bits) and an offset (low-order bits)
- The purpose of the page table is to map virtual pages onto page frames.

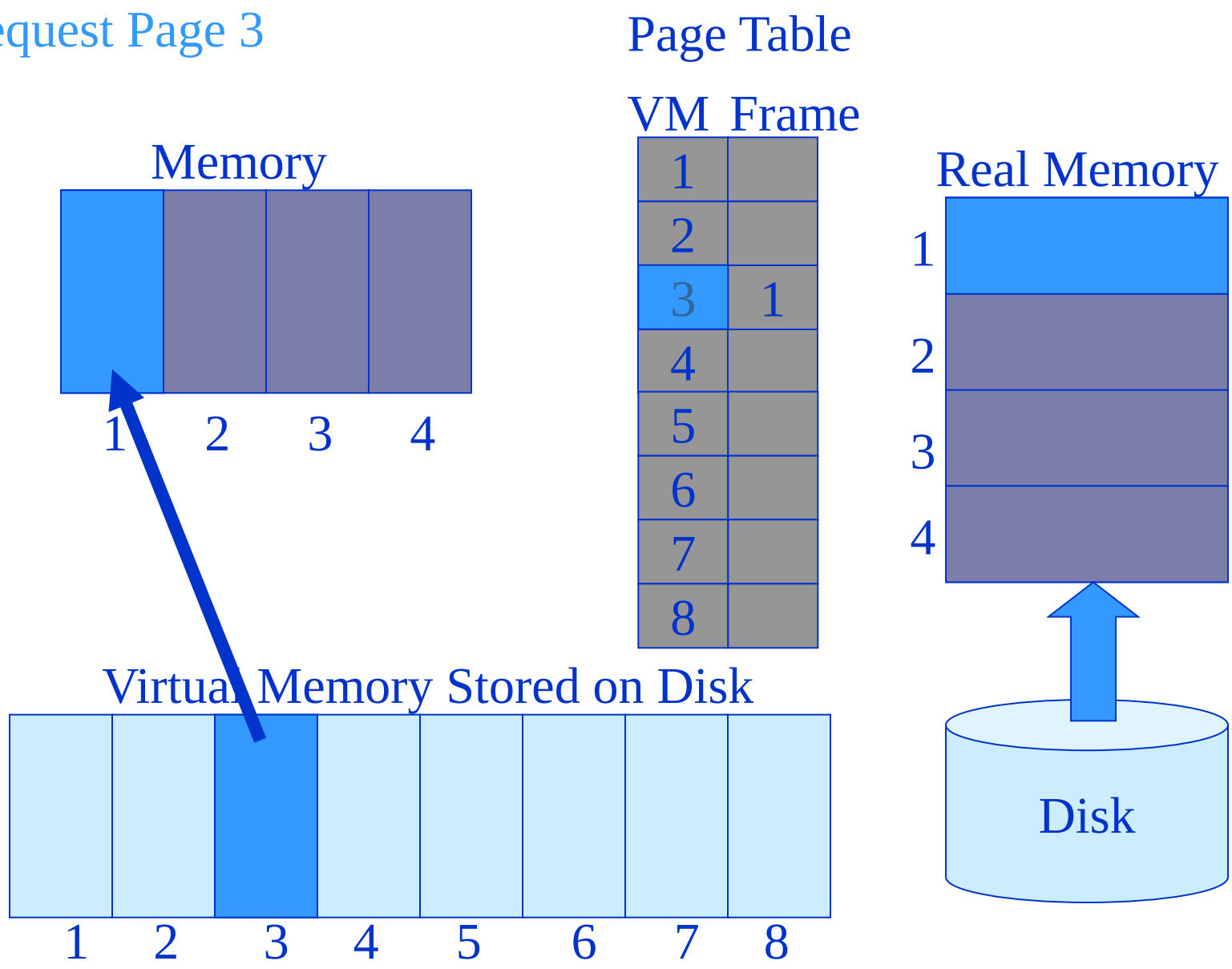
Typical page table entry



- Page frame number - map the frame number
- Present/absent bit - 1/0 indicates valid/invalid entry
- Protection bit - what kinds of access are permitted.
- Modified bit (**dirty bit**) - set when modified and writing to the disk occur
- Referenced bit - Set when page is referenced (help decide which page to evict)
- Caching disabled - Cache is used to keep data that logically belongs on the disk in memory to improve performance.

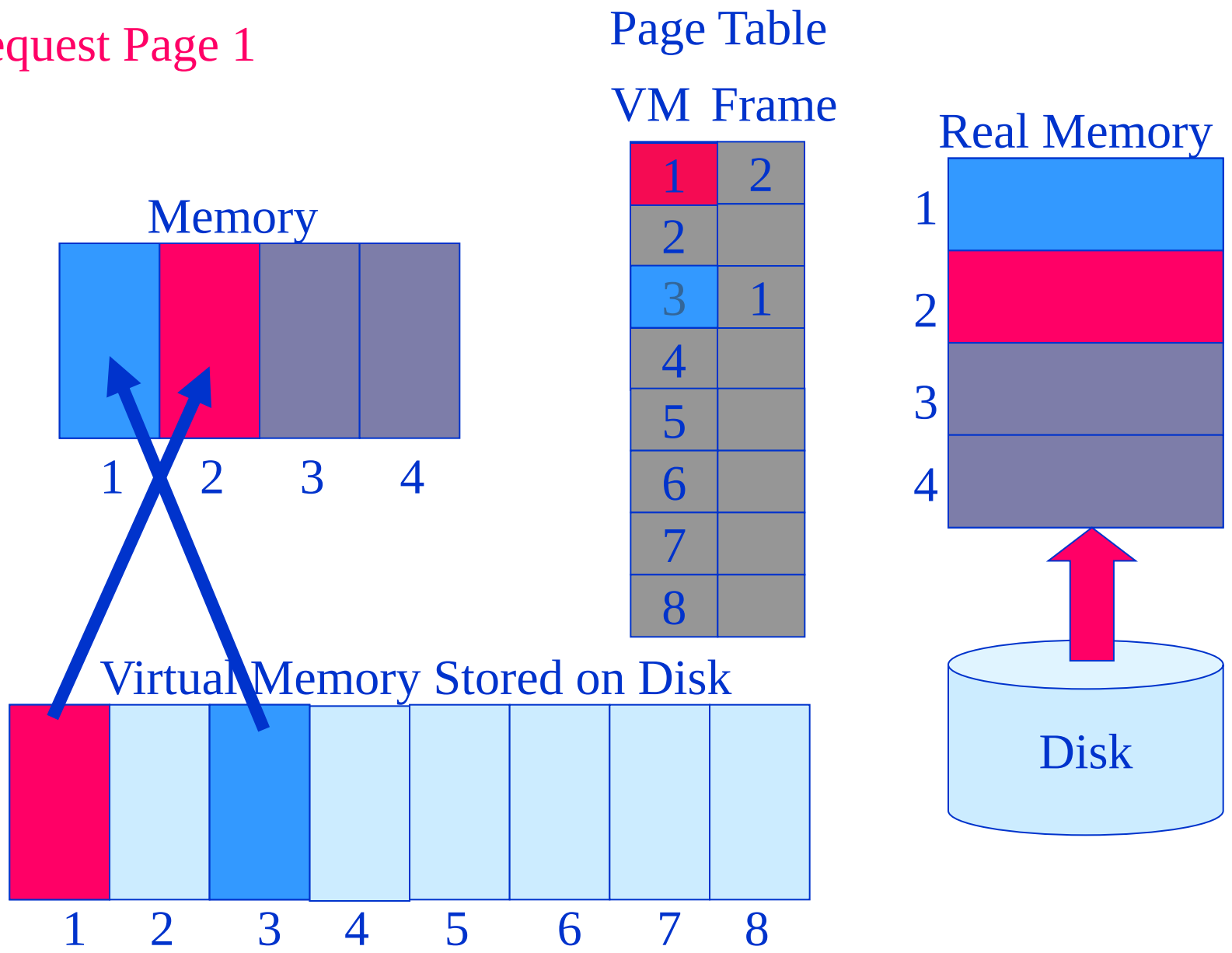
Paging Request

Request Page 3



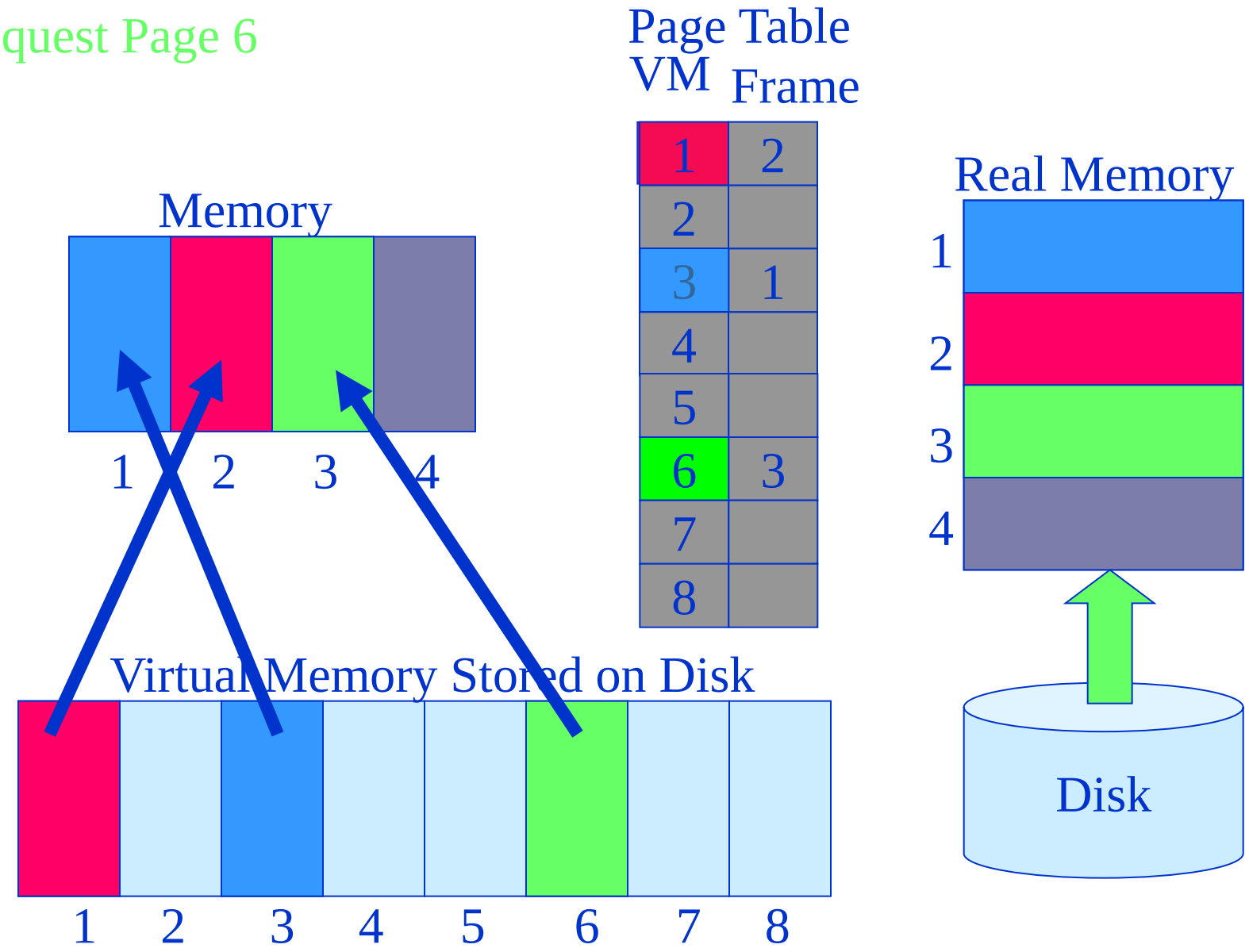
Paging

Request Page 1



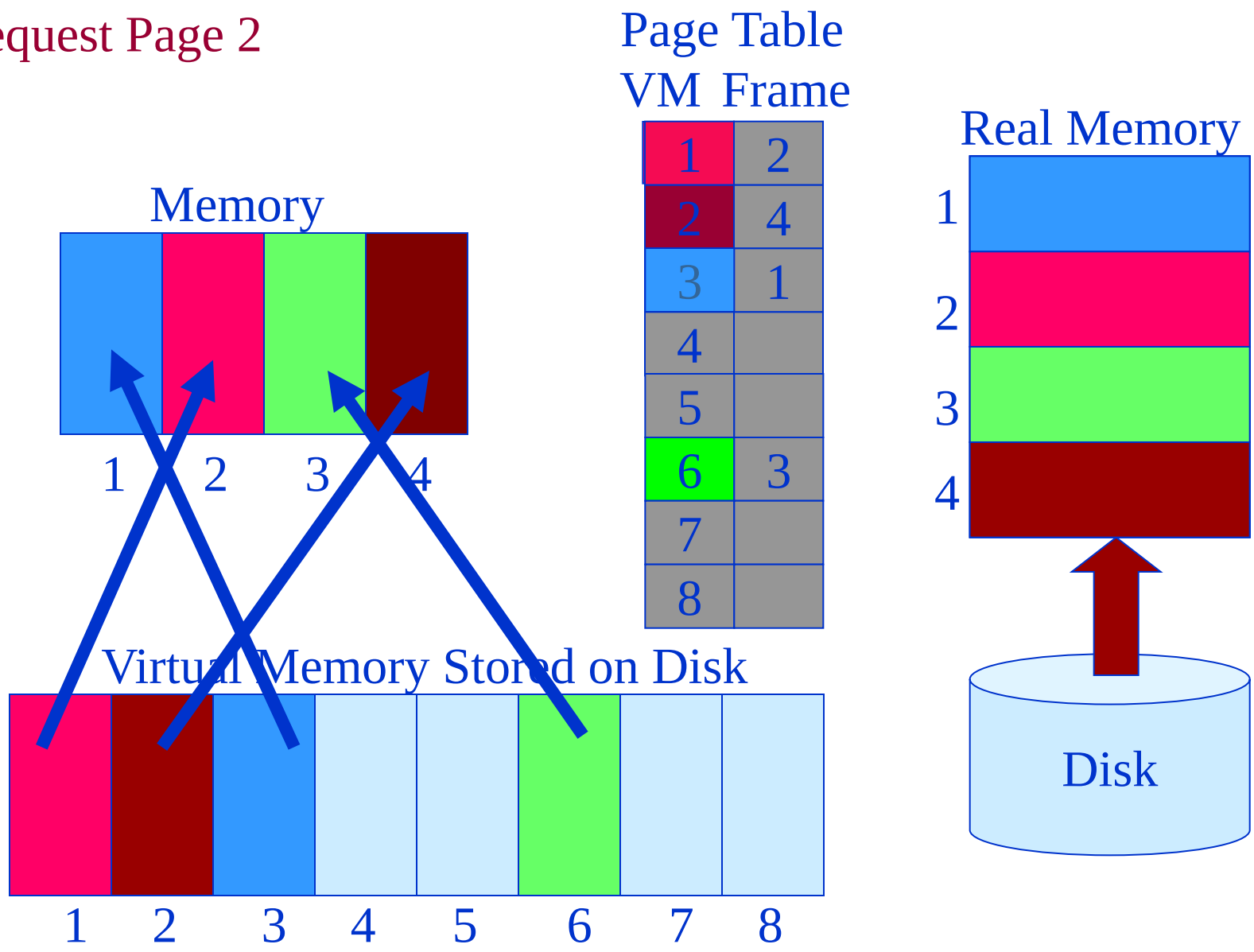
Paging

Request Page 6



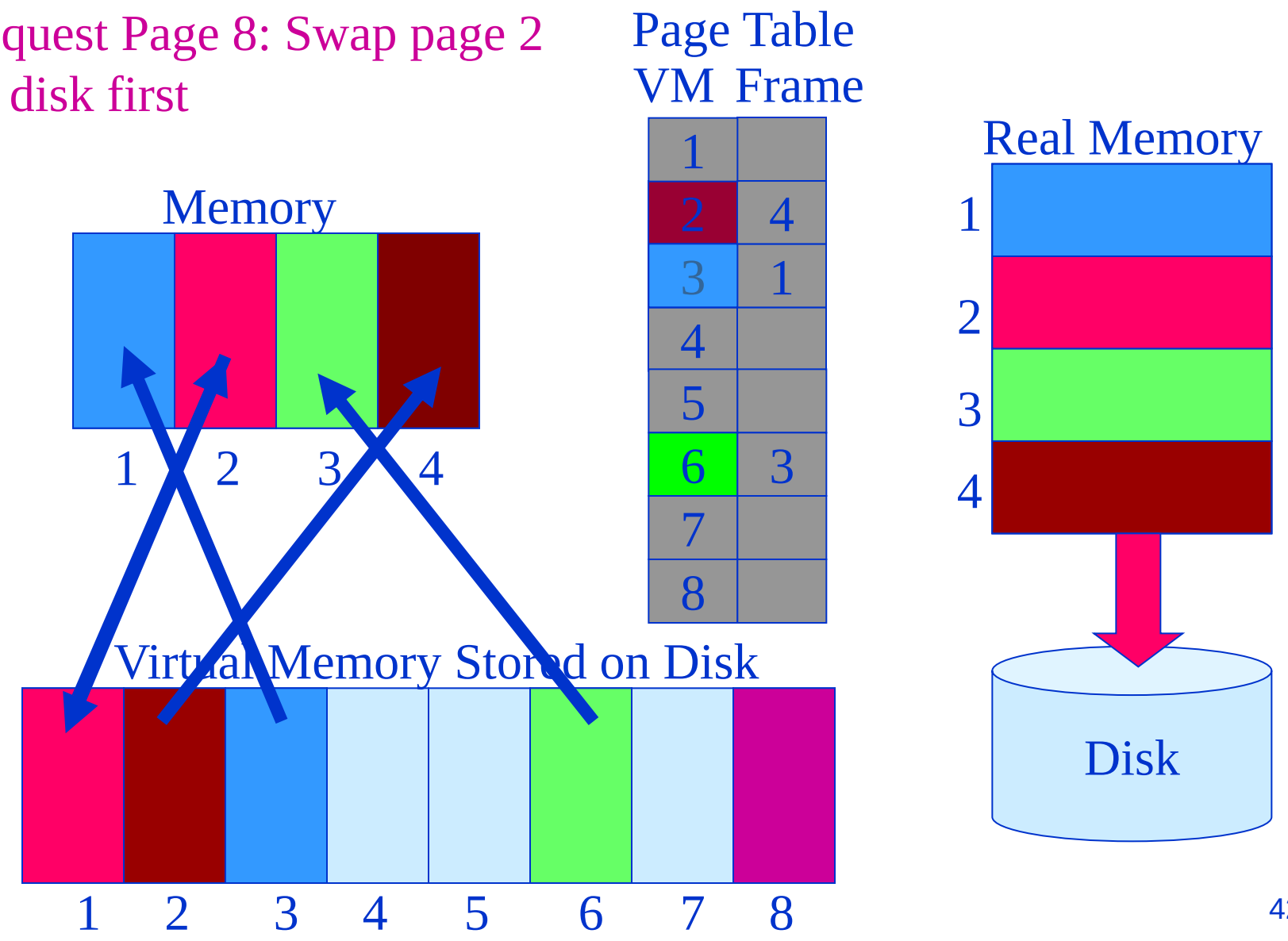
Paging

Request Page 2

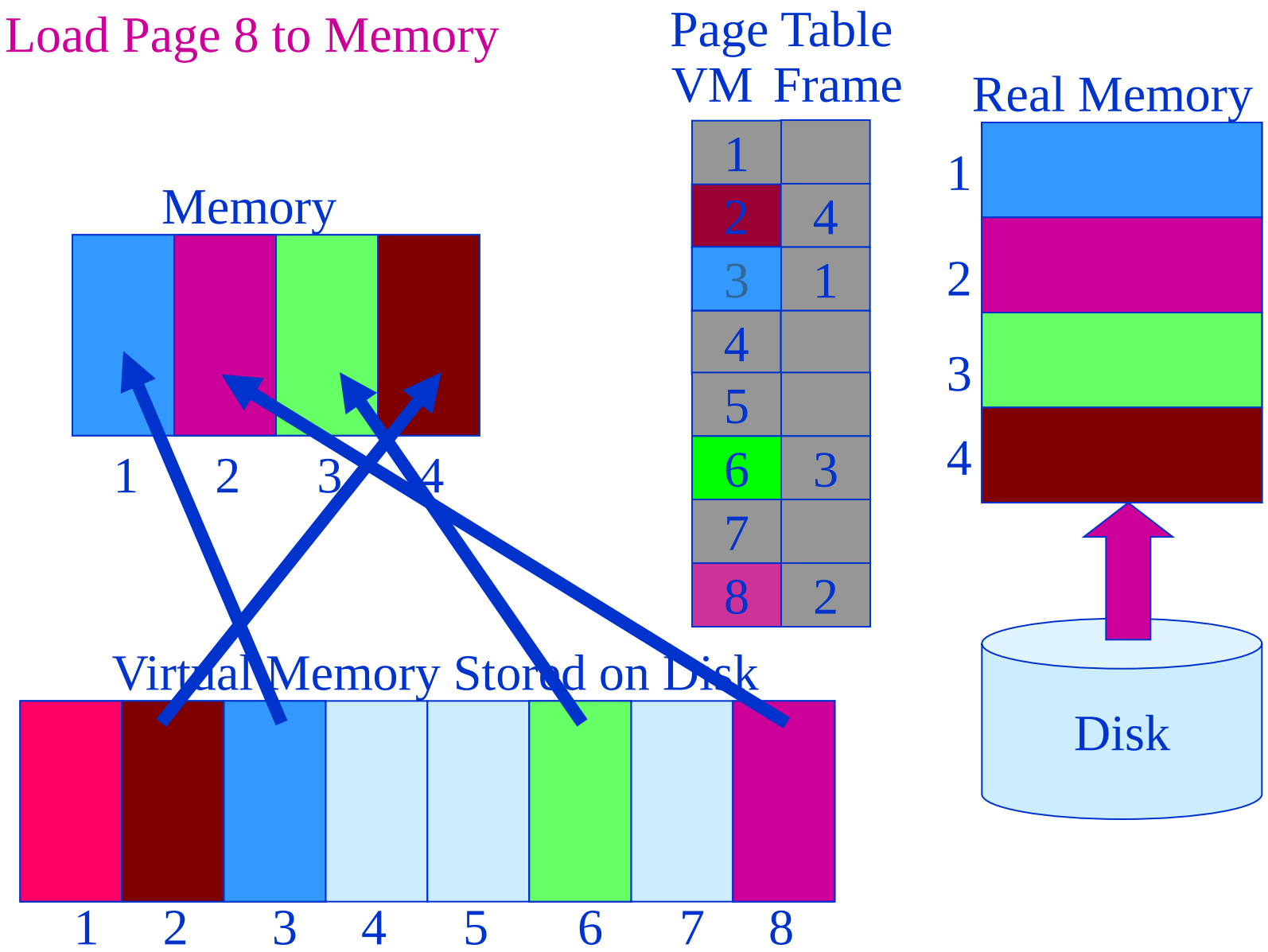


Paging

Request Page 8: Swap page 2 to disk first

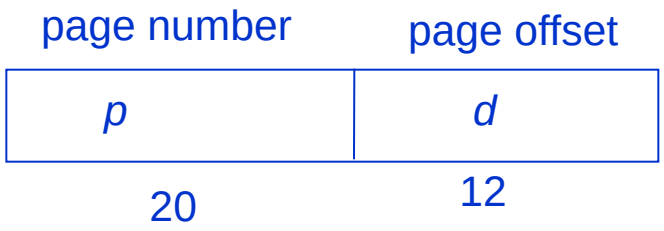


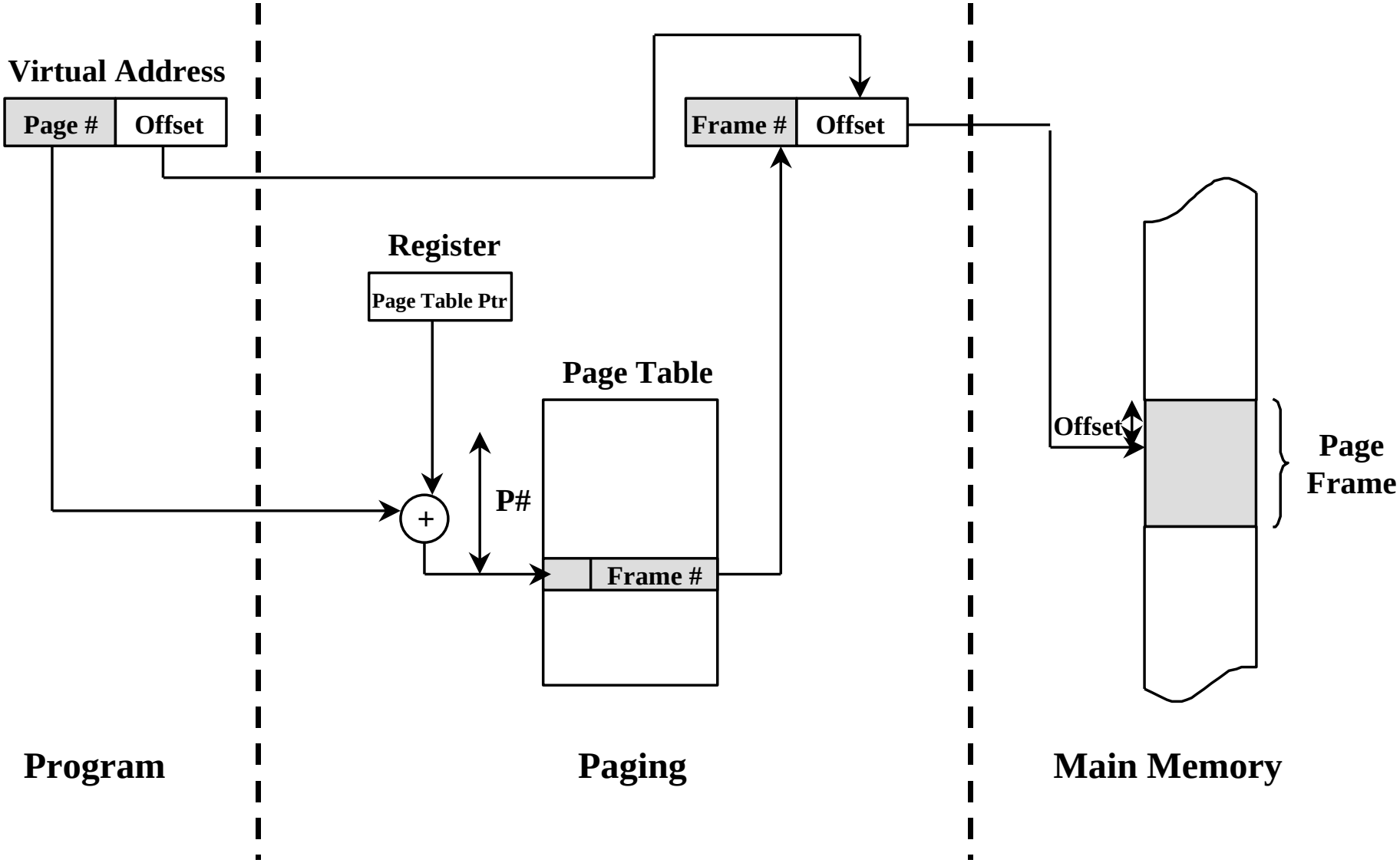
Paging


Load Page 8 to Memory
Page Table
VM Frame
Real Memory
Memory
1
2
3
4
1 4
2 1
3
4
5
6 3
7
8 2
1
2
3
4
Virtual Memory Stored on Disk
1 2 3 4 5 6 7 8
Disk

Paging Example

- A logical address (on 32-bit machine with 4K page size) is divided into:
  - a page number consisting of 20 bits.
  - a page offset consisting of 12 bits.
- Thus, a logical address is as follows:




Virtual Address
Page #
Offset
Register
Page Table Ptr
Page Table
P#
+
Frame #
Offset
Frame #
Offset
Page
Frame
Program
Paging
Main Memory

Page Tables Issues

- Two major issues of the page tables are faced:
  - The mapping must be fast because it is done on every memory access!!
  - Page tables may be extremely large (e.g. most computers use)

    32-bit address with 4k page size, 12-bit offset

    ➔20 bits for virtual page number

    ➔1 million entries!

Page Tables Issues

- Single page table consisting of an array of hardware registers. As a process is started up, the registers are loaded with page table.
  - Advantage - simple
  - Disadvantage - expensive if table is large and loading the full page table at every context switch hurts performance.
- Leave page table in memory - a single register points to the table
  - Advantage - context switch cheap
  - Disadvantage - one or more memory references to read table entries

Virtual-To-Physical Lookups

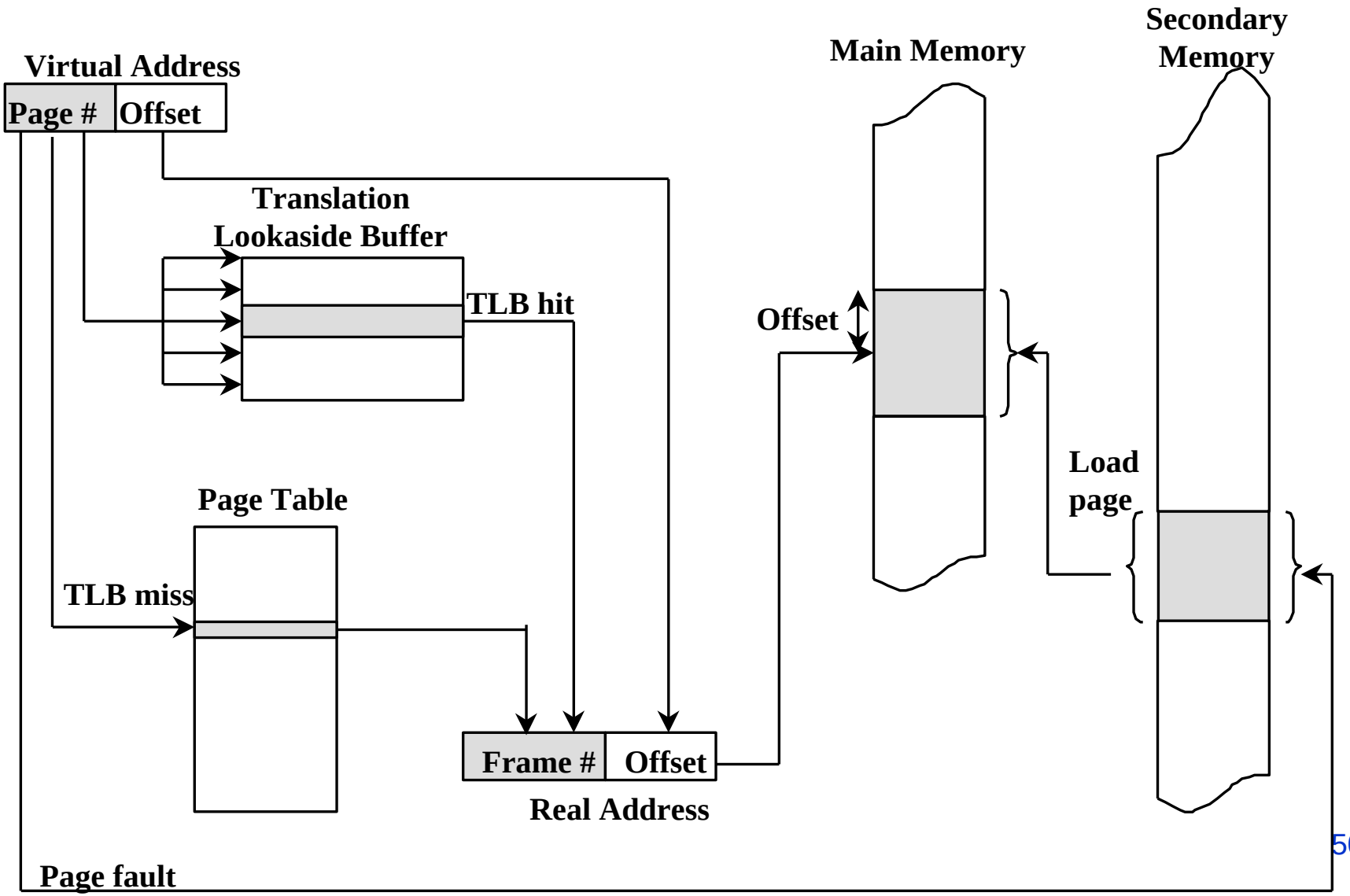
- Programs only know virtual addresses
- Each virtual address must be translated
  - May involve walking hierarchical page table
  - Page table stored in memory
  - So, each program memory access requires several actual memory accesses
- Solution: cache “active” part of page table
  - **Translation Look-aside Buffers *(TLBs)***
  - TLB also called “associative memory”

Bits in a TLB Entry

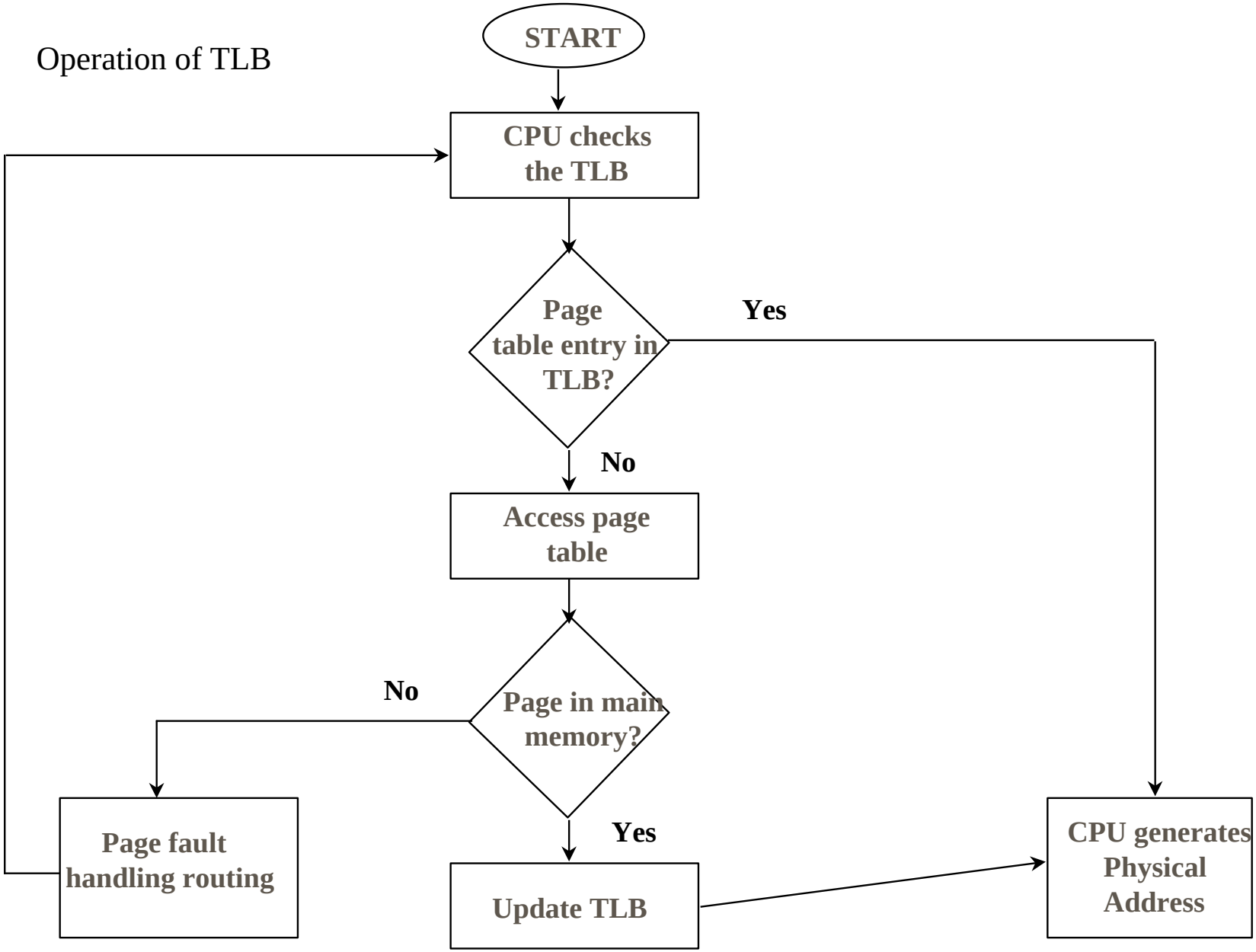
- Common (necessary) bits
  - Virtual page number: match with the virtual address
  - Physical page number: translated address
  - Valid
  - Access bits: kernel and user (nil, read, write)
- Optional (useful) bits
  - Process tag
  - Reference
  - Modify
  - Cacheable

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

Translation Look-aside Buffer (TLB)



Operation of TLB



TLB Function

- If a virtual address is presented to MMU, the hardware checks TLB by comparing all entries simultaneously (in parallel).
- If match is valid, the page is taken from TLB without going through page table.
- If match is not valid
  - MMU detects miss and does an ordinary page table lookup.
  - It then evicts one page out of TLB and replaces it with the new entry, so that next time that page is found in TLB.
- TLB hit ratio (Page address cache hit ratio)
  - Percentage of time page found in associative memory

Hardware-Controlled TLB

- On a TLB miss
  - Hardware loads the PTE (Page Table Entry) into the TLB
    - Need to write back if there is no free entry
  - Generate a fault if the page containing the PTE is invalid
  - VM software performs fault handling
  - Restart the CPU
- On a TLB hit, hardware checks the valid bit
  - If valid, pointer to page frame in memory
  - If invalid, the hardware generates a page fault
    - Perform page fault handling
    - Restart the faulting instruction

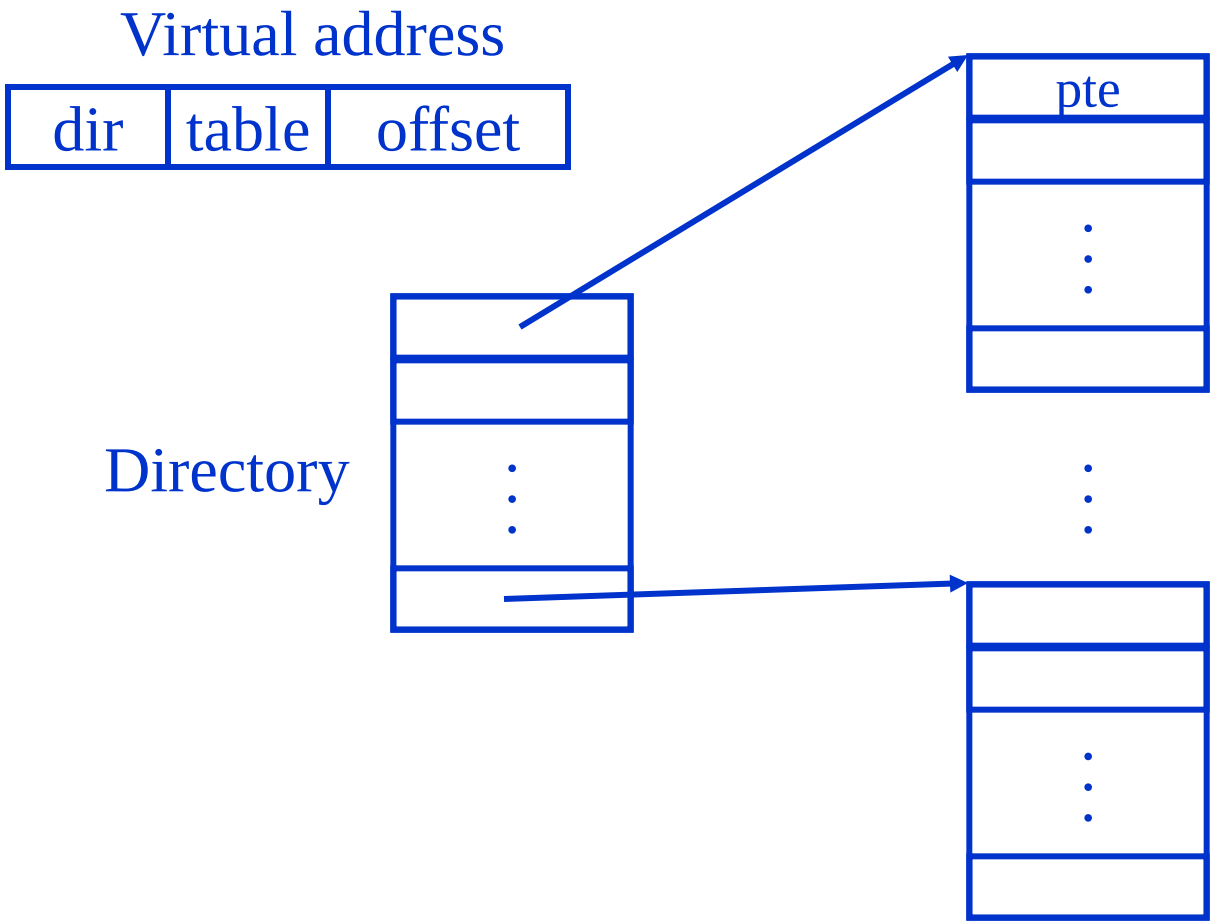
Software-Controlled TLB

- On a miss in TLB, generate a TLB fault, then trap to OS (software)
  - Check if the page containing the PTE is in memory
  - If no, perform page fault handling
  - Write back if there is no free entry, then load the PTE into the TLB
  - Restart the faulting instruction
- On a hit in TLB, the hardware checks valid bit
  - If valid, pointer to page frame in memory
  - If invalid, the hardware generates a page fault
    - Perform page fault handling
    - Restart the faulting instruction

Multilevel Page Tables

- Since the page table can be very large, one solution is to page the page table
- Divide the page number into
  - An index into a page table of second level page tables
  - A page within a second level page table
- Advantage:
  - reduce the table size.
  - don't keep page tables in memory that are not needed.

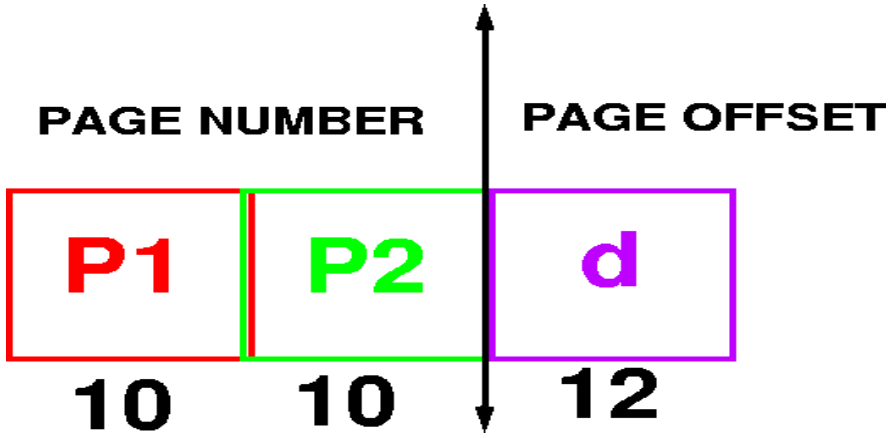
Multiple-Level Page Tables

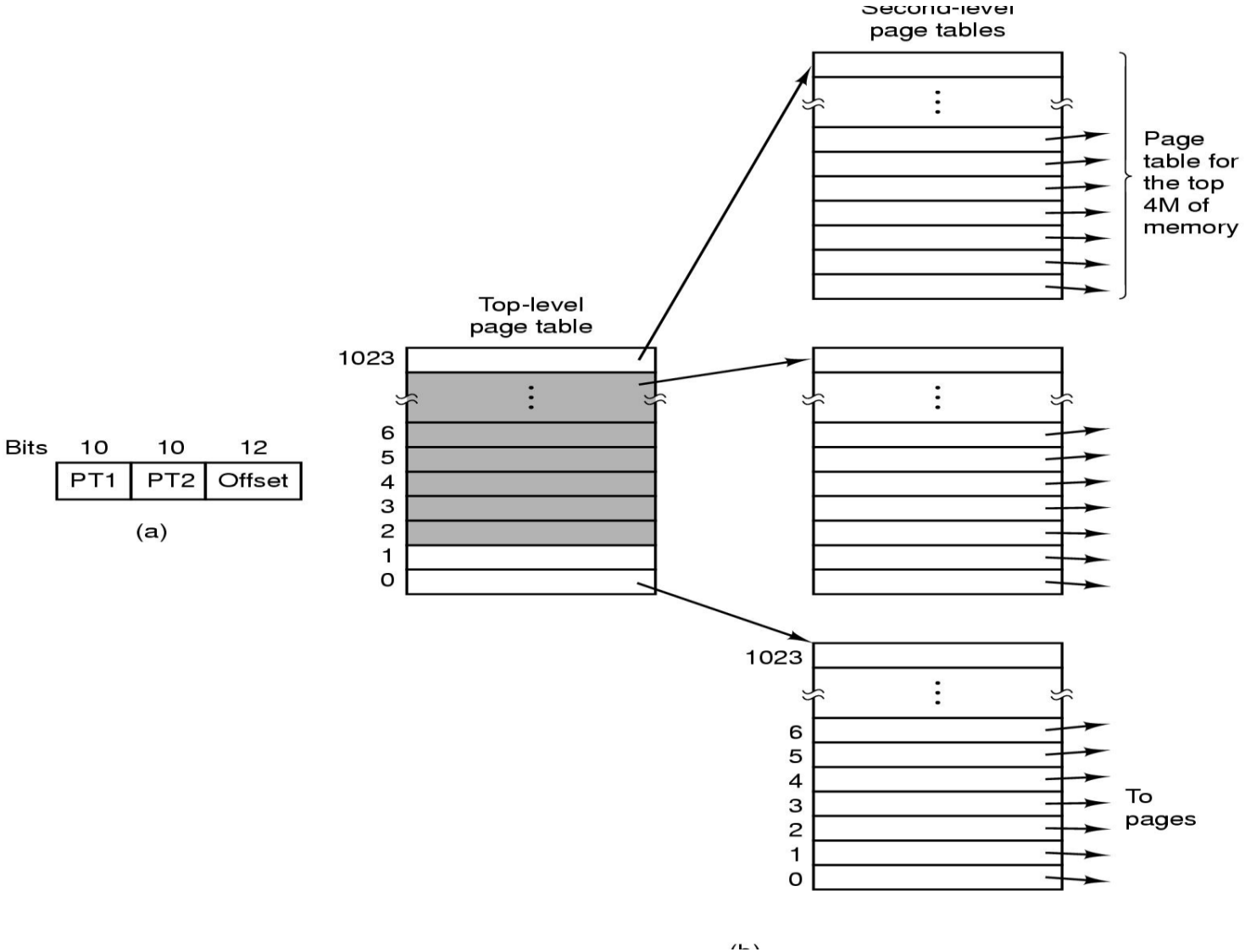


What does this by us? Sparse address spaces and easier paging

Example Addressing on a Multilevel Page Table System

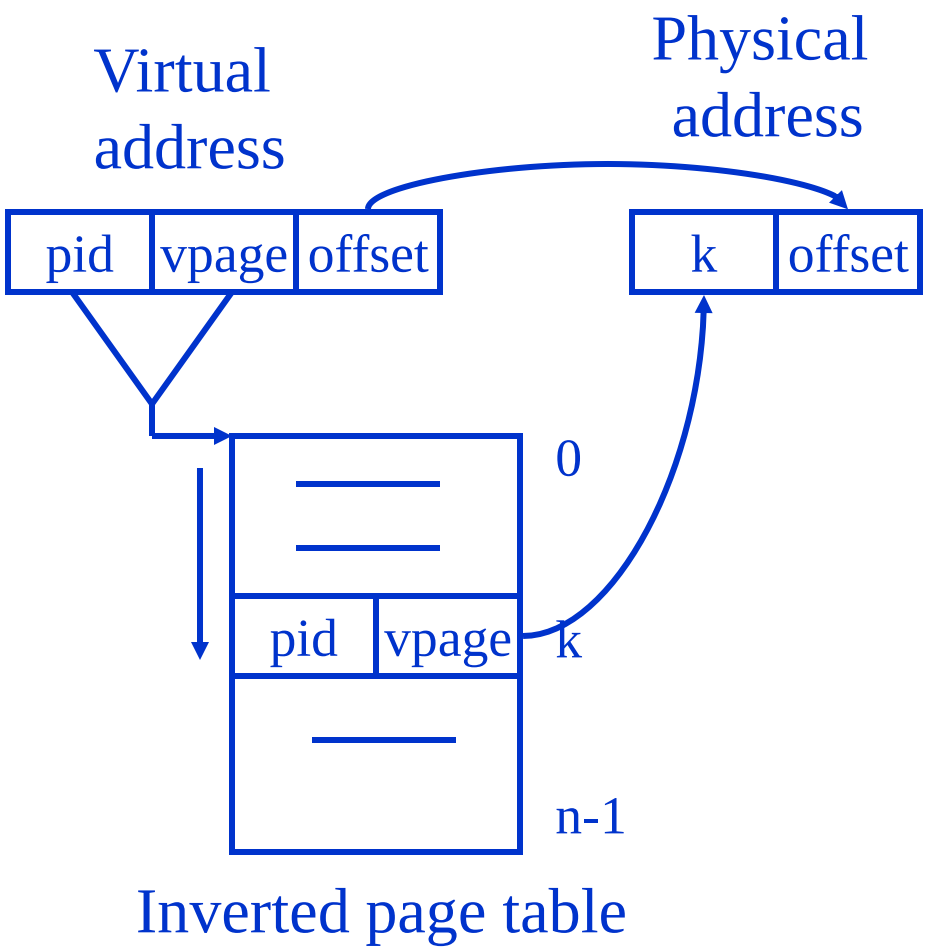
- A logical address (on 32 bit machine with 4k page size) is divided into
  - A page number consisting of 20 bits
  - A page offset consisting of 12 bits
- Divide the page number into
  - A 10-bit page number
  - A 10-bit page offset





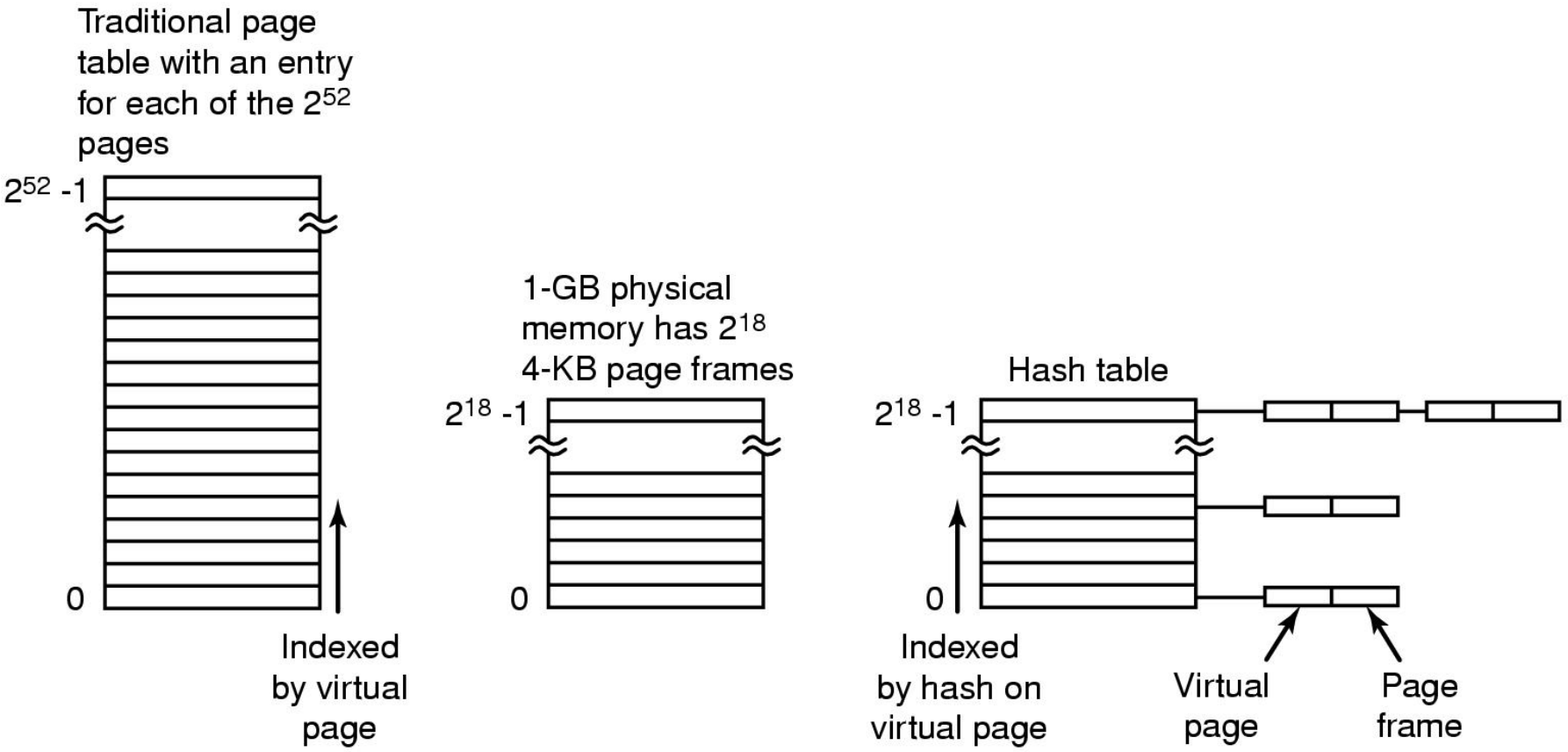
Performance: Since each level is stored as a separate table in memory, converting a logical address to a physical one in a four-level paging may take five memory accesses.

Inverted Page Tables



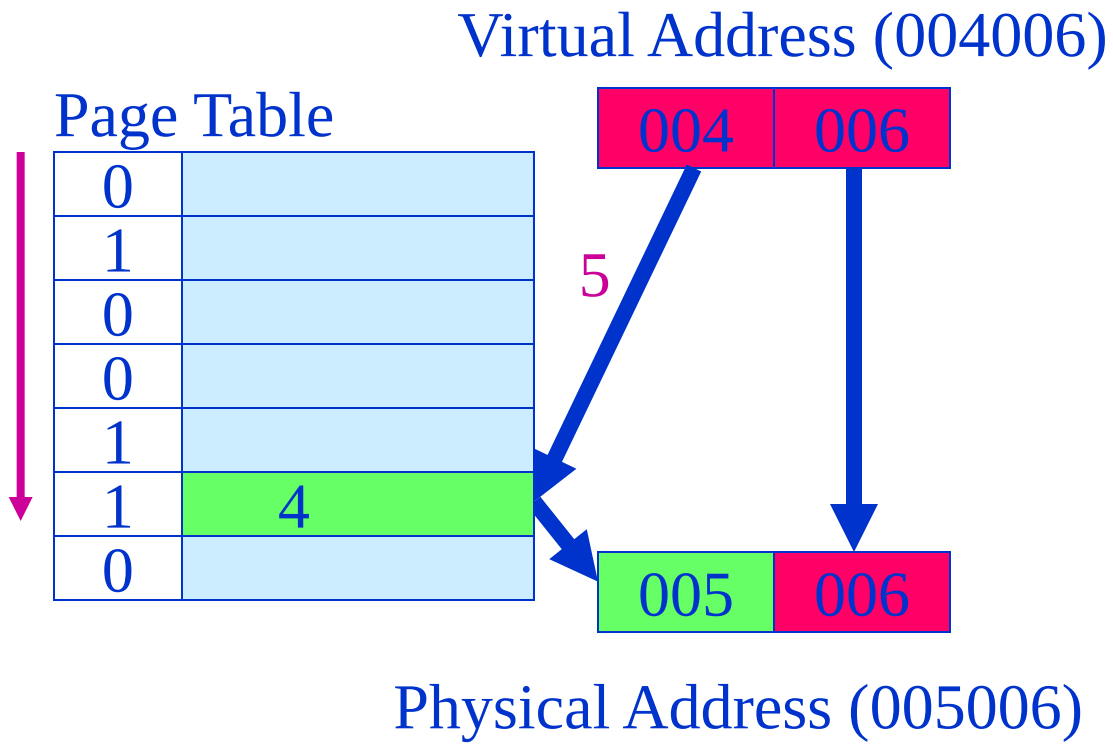
- Main idea
  - One PTE for each physical page frame
  - Hash (Vpage, pid) to Ppage#
- Pros
  - Small page table for large address space
- Cons
  - Lookup is difficult
  - Overhead of managing hash chains, etc

Inverted Page Tables



Comparison of traditional page table with an inverted page table

Inverted Page Table



Effective Access Time

- Associative Lookup = ε time unit
- Assume memory cycle time is t time unit
- Hit ratio – percentage of times that a page number is found in the associative registers; ration related to number of associative registers.
- Hit ratio = α
- Effective Access Time (EAT)

$$EAT = \alpha\,(t + \varepsilon) + (1 - \alpha)\,(2t + \varepsilon)$$
$$= \alpha\, t + \alpha\, \varepsilon + 2t + \varepsilon - 2\alpha t - \alpha\, \varepsilon$$
$$= (2 - \alpha)t + \varepsilon$$

- Example: $\alpha = 0.8$, $\varepsilon = 20$ ns, $t = 100$ ns

  EAT = 0.8 x 120 + 0.2 x (200 + 20) = 140 ns.

Inverted Page Table

- Usually, each process has a page table associated with it. One of drawbacks of this method is that each page table may consist of millions of entries.
- To solve this problem, an **inverted page table** can be be used. There is one entry for each real page (frame) of memory.
- Each entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page.
- Examples of systems using the inverted page tables include 64-bit UltraSPARC and PowerPC.

Inverted Page Table

- To illustrate this method, a simplified version of the implementation of the inverted page is described as: <process-id, page-number, offset>.
- Each inverted page-table entry is a pair <process-id, page-number>. The inverted page table is then searched for a match. If a match i found, then the physical address <i, offset> is generated. Otherwise, an illegal address access has been attempted.
- Although it decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs.
- Use hash table to limit the search to one — or at most a few — page-table entries.

Inverted Page Table Architecture

